ZELFSTUDIECURSUS

Angular 22

Weekmenu: recepten, weekplanning en boodschappenlijst

Signals, routing, formulieren, RxJS, HTTP, auth, tests
Tailwind CSS v4, Supabase, Vercel

Inhoudsopgave

Inleiding **4**

Deel 1: Project opzetten, structuur en je eerste signal **6**

MODULE 1: Node.js controleren en een nieuw Angular-project maken 6
MODULE 2: De projectstructuur begrijpen 7
MODULE 3: Opruimen en Tailwind controleren 9
MODULE 4: Templates en je eerste signal 9

Deel 2: Templates, control flow en componenten **12**

MODULE 5: Het datamodel en de hulpfuncties 12
MODULE 6: Een herbruikbaar component met input() en output() 14
MODULE 7: Content projection met ng-content 16
MODULE 8: Een lijst met filter, @for en computed() 17

Deel 3: Services en dependency injection **20**

MODULE 9: Wat is een service en wat is dependency injection? 20
MODULE 10: De RecipeStore 20
MODULE 11: De service gebruiken met inject() 21

Deel 4: Routing **23**

MODULE 12: Pagina's, routes en het app-shell 23
MODULE 13: De lijstpagina, de detailpagina en routeparameters 25

Deel 5: Formulieren met Signal Forms **30**

MODULE 14: Welke formulieren heeft Angular? 30
MODULE 15: Een herbruikbare foutmelding 30
MODULE 16: Het receptformulier 31

Deel 6: RxJS, Reactive Forms en effecten **39**

MODULE 17: Wat is RxJS en waarom kom je het overal tegen? 39
MODULE 18: Een zoekveld met Reactive Forms en RxJS 40

Deel 7: Supabase en authenticatie **44**

MODULE 19: Een Supabase-project met beveiligde tabellen 44
MODULE 20: Environments en de AuthStore 46
MODULE 21: De loginpagina met Reactive Forms 48
MODULE 22: Route guards en de header 51

Deel 8: HTTP, httpResource en interceptors **55**

MODULE 23: HttpClient en interceptors 55
MODULE 24: De twee interceptors 55
MODULE 25: De RecipeStore met httpResource 57
MODULE 26: De componenten aanpassen 59

Deel 9: De weekplanner en de boodschappenlijst **67**

MODULE 27: Het model en de rekenlogica 67
MODULE 28: De PlanStore 68
MODULE 29: De pagina Weekplanner 70
MODULE 30: De boodschappenlijst 72

Deel 10: Pipes, directives en @defer **77**

MODULE 31: Pipes 77
MODULE 32: Directives 79
MODULE 33: @defer: onderdelen later laden 82
MODULE 34: Levenscyclus en template-referenties 85

Deel 11: Testen met Vitest 87
MODULE 35: Hoe test je Angular? 87
MODULE 36: Pure functies en pipes testen 87
MODULE 37: Een component en een service testen 90

Deel 12: Bouwen, SSR en deployen 93
MODULE 38: De productiebuild 93
MODULE 39: Server-side rendering (SSR): wat, wanneer, en niet voor deze app 94
MODULE 40: Deployen naar Vercel 94

Bijlage A: Oudere Angular-code herkennen 97

Bijlage B: Vue/React naar Angular: spiekbriefje 99

Bijlage C: Veelgemaakte fouten 101

Bijlage D: Eindopdrachten en wat je daarna leert 102
Eindopdrachten 102
Wat leer je daarna? 103

Bijlage E: Overzicht van het eindresultaat 104

Bijlage F: Commando-overzicht 105

Inleiding

Deze cursus is bedoeld om je zelfstandig te begeleiden, op dezelfde manier als de Vue- en React-cursus. Je leest de tekst, volgt de stappen in volgorde, typt de code in je editor en controleert telkens het effect in je browser. Je hebt geen docent nodig. Ook deze cursus legt uit:

- exact wat je moet doen,
- waarom je het doet,
- en wat de belangrijke regels code betekenen.

Wat bouw je?

Je bouwt **Weekmenu**, een app waarin je:

- je eigen **recepten** beheert (met ingrediënten, bereidingstijd en aantal personen),
- een recept **omrekent** naar een ander aantal personen,
- je recepten over de **week** verdeelt (lunch en diner per dag),
- automatisch een **boodschappenlijst** krijgt met alle ingrediënten van die week,
- **inlogt** met een eigen account, zodat ieders recepten privé zijn.

Dit project dwingt vrijwel alle onderdelen af die je in een echte Angular-applicatie nodig hebt: componenten, signals, services, routing, formulieren, RxJS, HTTP, interceptors, guards, authenticatie, pipes, directives, tests en deployment.

Wat leer je in deze cursus?

Deel	Onderwerp
1	Angular CLI, projectstructuur, Tailwind, je eerste component en signals
2	Templates, control flow (`@if`, `@for`, `@switch`), `input()`, `output()`, content projection
3	Services en dependency injection
4	Routing: pagina's, parameters, lazy loading
5	Formulieren met Signal Forms (met een dynamische lijst ingrediënten)
6	RxJS, Reactive Forms, `effect()` en `linkedSignal()`
7	Supabase, authenticatie met Reactive Forms en route guards
8	HTTP, `httpResource`, interceptors en de echte database
9	De weekplanner en de boodschappenlijst
10	Pipes, directives en `@defer`
11	Testen met Vitest
12	Productiebuild, SSR-uitleg en deployen naar Vercel
Bijlagen	Oudere Angular-code herkennen, spiekbriefje, veelgemaakte fouten, eindopdrachten

Wat heb je nodig?

- **Node.js 22.22.3 of nieuwer** (aanbevolen: de huidige LTS, Node 24). Angular 22 weigert te starten op een oudere versie.
- Visual Studio Code met de extensie *Angular Language Service*.
- Basiskennis van HTML, CSS en TypeScript. Kennis van Vue of React helpt: in de tekst staan blokken **Vergelijking met Vue/React**.
- Voor Deel 7 en verder: een gratis account op supabase.com, github.com en vercel.com.

Afspraken in deze cursus

- Alle code is **TypeScript**. De bestandsnamen volgen de **stijlgids van 2025**: een component heet `recipe-card.ts` met de klasse `RecipeCard` (zonder `.component` in de naam). Oudere tutorials en bedrijfscode gebruiken nog `recipe-card.component.ts`; dat staat uitgelegd in Bijlage A.
- Bij elk codeblok staat het **exacte bestandspad** boven de code. Je plakt altijd het **volledige bestand**, tenzij er expliciet staat dat je iets onderaan toevoegt.
- Commando's voer je uit in de terminal van VS Code in de projectmap.
- Na elk deel kun je met `npm run build` controleren of alles compileert.

Deel 1: Project opzetten, structuur en je eerste signal

MODULE 1: Node.js controleren en een nieuw Angular-project maken

Doel van deze module

Na deze module heb je een draaiend Angular 22-project met Tailwind CSS v4.

Stap 1: Node.js controleren

```
node -v
npm -v
```

Toelichting

- Je hebt minstens `v22.22.3` nodig (of `v24.15.0` en hoger). Heb je een oudere versie, dan toont de Angular CLI een melding en stopt hij. Installeer in dat geval de LTS-versie via nodejs.org.

Stap 2: Een cursusmap aanmaken

```
mkdir angular2025
cd angular2025
```

Stap 3: Het project genereren met de Angular CLI

```
npx @angular/cli@latest new weekmenu --style=tailwind --routing --ssr=false --ai-config=none
```

Toelichting van dit commando

- `npx @angular/cli@latest new weekmenu` start de nieuwste Angular CLI zonder hem permanent te installeren, en maakt een project met de naam `weekmenu`.
- `--style=tailwind` zet Tailwind CSS v4 meteen goed (inclusief de PostCSS-configuratie). Je hoeft niets handmatig te installeren.
- `--routing` maakt het bestand voor routes aan.
- `--ssr=false` zet server-side rendering uit. Dat bespreken we in Deel 12; eerst bouwen we een gewone browser-applicatie.
- `--ai-config=none` slaat configuratiebestanden voor AI-assistenten over.
- Laat je deze opties weg, dan stelt de CLI ze als vragen. Voor de rest laat je de standaardkeuzes staan.

Dit commando installeert ook alle pakketten. Dat duurt even.

Stap 4: Starten

```
cd weekmenu
npm start
```

Toelichting

- `npm start` voert `ng serve` uit: de ontwikkelserver. Open `http://localhost:4200`. Je ziet de welkomstpagina van Angular.
- Bij elke opgeslagen wijziging herlaadt de pagina automatisch.
- Stoppen kan met `Ctrl + C`. Laat de server draaien en open een **tweede terminal** voor andere commando's.
- Wil je het commando `ng` rechtstreeks gebruiken? Installeer de CLI globaal met `npm install -g @angular/cli`. In deze cursus gebruiken we `npx ng ...`, wat altijd werkt.

MODULE 2: De projectstructuur begrijpen

Doel van deze module

Na deze module weet je welke bestanden er zijn, wat ze doen en hoe een Angular-app opstart.

```
weekmenu/
  angular.json            configuratie van de CLI: build, serve, test
  package.json            pakketten en scripts
  tsconfig*.json          TypeScript-instellingen
  .postcssrc.json         Tailwind via PostCSS
  public/                 statische bestanden (favicon, afbeeldingen)
  src/
    index.html            de enige HTML-pagina
    main.ts               het startpunt
    styles.css            globale stijlen (hier staat Tailwind)
    app/
      app.ts              het root-component (klasse App)
      app.html            de template van het root-component
      app.config.ts       providers van de hele applicatie
      app.routes.ts       de routes (nu nog leeg)
```

src/index.html en src/main.ts

`index.html` bevat `<app-root></app-root>`. Dat is de plek waar Angular zijn applicatie tekent. `main.ts` start dat op:

```
import { bootstrapApplication } from '@angular/platform-browser';
import { appConfig } from './app/app.config';
import { App } from './app/app';

bootstrapApplication(App, appConfig)
  .catch((err) => console.error(err));
```

Toelichting

- `bootstrapApplication(App, appConfig)` start de applicatie met `App` als **root-component** en `appConfig` als configuratie.

- Alle componenten in Angular zijn **standalone**: ze regelen zelf wat ze nodig hebben via hun `imports`-lijst. Er zijn geen `NgModule`s meer nodig (die zie je nog wel in oudere code, zie Bijlage A).

src/app/app.config.ts

```
import { ApplicationConfig, provideBrowserGlobalErrorListeners } from '@angular/core';
import { provideRouter } from '@angular/router';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideRouter(routes)
  ]
};
```

Toelichting

- `providers` is de lijst met voorzieningen voor de hele app: de router, later HTTP, interceptors en meer. Je voegt functies zoals `provideRouter(...)` en `provideHttpClient(...)` hier toe.
- Je ziet geen `zone.js`: nieuwe Angular-projecten zijn **zoneless**. Angular tekent alleen opnieuw als een **signal** verandert (zie Module 4). Dat is de moderne manier en veel voorspelbaarder dan het oude systeem.

Een component

Een component is een **klasse met een decorator**. Zo ziet het root-component eruit:

```
@Component({
  imports: [RouterOutlet],
  selector: 'app-root',
  templateUrl: './app.html',
})
export class App { }
```

- `selector` is de HTML-tag waarmee je het component gebruikt (`<app-root>`).
- `templateUrl` verwijst naar het HTML-bestand. (Voor kleine componenten mag je ook `template: '...'` direct in de klasse zetten.)
- `imports` bevat alles wat de template gebruikt: andere componenten, pipes en directives.

> **Vergelijking met Vue/React:** Vue heeft één `.vue`-bestand per component, React een functie. Angular splitst een component in **klasse (gedrag)** en **template (HTML)**, in aparte bestanden. De klasse is bovendien geen functie die telkens opnieuw draait (React): ze wordt **één keer** aangemaakt en blijft bestaan.

MODULE 3: Opruimen en Tailwind controleren

Stap 1: Overbodige bestanden verwijderen

```
del src\app\app.css
del src\app\app.spec.ts
```

Toelichting: `app.css` is leeg (we gebruiken Tailwind), en `app.spec.ts` is de voorbeeldtest van de welkomstpagina die we vervangen. In Deel 11 schrijf je zelf tests.

Stap 2: src/styles.css controleren

Het bestand moet dit bevatten (de CLI heeft het al goed gezet):

```
@import 'tailwindcss';
```

Stap 3: index.html in het Nederlands

Bestand: src/index.html

```
<!doctype html>
<html lang="nl">
<head>
  <meta charset="utf-8">
  <title>Weekmenu</title>
  <base href="/">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="icon" type="image/x-icon" href="favicon.ico">
</head>
<body>
  <app-root></app-root>
</body>
</html>
```

Toelichting

- `<base href="/">` is nodig voor de router: het vertelt de browser vanaf welk pad relatieve links werken.
- `lang="nl"` geeft de taal van de pagina aan.

MODULE 4: Templates en je eerste signal

Doel van deze module

Na deze module begrijp je:

- wat een **signal** is,
- hoe je een waarde toont (interpolatie) en op een klik reageert (event binding),
- hoe Angular de pagina bijwerkt.

Stap 1: app.ts

Bestand: src/app/app.ts

```ts
import { Component, signal } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.html',
})
export class App {
  protected readonly count = signal(0);

  protected addOne(): void {
    this.count.update((value) => value + 1);
  }
}
```

Stap 2: app.html

Bestand: src/app/app.html

```html
<div class="min-h-screen bg-stone-50">
  <main class="mx-auto max-w-md p-8">
    <h1 class="text-3xl font-bold text-orange-700">Weekmenu</h1>

    <p class="mt-2 text-stone-600">
      Je hebt {{ count() }} {{ count() === 1 ? 'recept' : 'recepten' }} gekookt.
    </p>

    <button
      type="button"
      (click)="addOne()"
      [disabled]="count() >= 10"
      class="mt-4 rounded-lg bg-orange-600 px-4 py-2 font-medium text-white hover:bg-orange-700
disabled:opacity-50"
    >
      Gekookt!
    </button>
  </main>
</div>
```

Toelichting: de klasse

- `signal(0)` maakt een **signal** met beginwaarde `0`. Een signal is een "doos met een waarde" die Angular in de gaten houdt. Je **leest** hem door hem als functie aan te roepen: `count()`.
- Je **wijzigt** hem met `set(nieuweWaarde)` of `update((oud) => nieuw)`. Hier gebruiken we `update`, want de nieuwe waarde hangt van de oude af.
- `protected` betekent: zichtbaar voor de template, maar niet voor andere klassen. `readonly` betekent dat de verwijzing naar de signal zelf nooit verandert (de waarde in de signal wel).

Toelichting: de template

- `{{ count() }}` is **interpolatie**: toont een waarde als tekst. Je schrijft `count()` met haakjes omdat je de signal leest.
- `(click)="addOne()"` is **event binding**: haakjes rond de naam van een DOM-event. Bij een klik voert Angular de methode uit.
- `[disabled]="count() >= 10"` is **property binding**: vierkante haken binden een DOM-eigenschap aan een expressie. Zodra `count()` 10 is, wordt de knop uitgeschakeld.

- De Tailwind-klassen staan gewoon in `class="..."`.

Belangrijk inzicht: hoe Angular de pagina bijwerkt

1. Je klikt, `addOne()` roept `count.update(...)` aan.
2. Angular weet **welke delen van de template** `count()` lezen.
3. Alleen die delen worden bijgewerkt. Er wordt niet de hele pagina doorgerekend.

> **Vergelijking met Vue/React:** een signal lijkt sterk op een `ref` in Vue (maar je schrijft `count()` in plaats van `count.value`) en op `useState` in React (maar de klasse draait niet opnieuw: alleen de template-delen die de signal lezen worden bijgewerkt).

Deel 2: Templates, control flow en componenten

MODULE 5: Het datamodel en de hulpfuncties

Doel van deze module

Een bestand met de types voor recepten, hulpfuncties (puur TypeScript, geen Angular) en voorbeeldrecepten.

Stap 1: De mappen aanmaken

```
mkdir src\app\recipes src\app\shared src\app\core
```

Stap 2: src/app/recipes/recipe.ts

Bestand: src/app/recipes/recipe.ts

```
export const CATEGORIES = ['ontbijt', 'lunch', 'diner', 'dessert'] as const;
export type Category = (typeof CATEGORIES)[number];

export const UNITS = ['g', 'ml', 'stuk', 'el', 'tl'] as const;
export type Unit = (typeof UNITS)[number];

export interface Ingredient {
  name: string;
  amount: number;
  unit: Unit;
}

export interface Recipe {
  id: number;
  title: string;
  description: string;
  category: Category;
  servings: number;
  minutes: number;
  ingredients: Ingredient[];
  steps: string;
}

// Een nieuw recept heeft nog geen id.
export type NewRecipe = Omit<Recipe, 'id'>;

// Reken een hoeveelheid om van het basisaantal personen naar een ander aantal.
export function scaleAmount(amount: number, baseServings: number, servings: number): number {
  const scaled = (amount * servings) / baseServings;
  return Math.round(scaled * 100) / 100;
}

export function formatAmount(amount: number, unit: Unit): string {
  const text = new Intl.NumberFormat('nl-BE', { maximumFractionDigits: 2 }).format(amount);
  if (unit === 'stuk') return `${text} ${amount === 1 ? 'stuk' : 'stuks'}`;
  return `${text} ${unit}`;
}

export const SAMPLE_RECIPES: Recipe[] = [
  {
```

```
    id: 1,
    title: 'Pannenkoeken',
    description: 'Dunne pannenkoeken voor een lui zondagontbijt.',
    category: 'ontbijt',
    servings: 4,
    minutes: 25,
    ingredients: [
      { name: 'Bloem', amount: 250, unit: 'g' },
      { name: 'Melk', amount: 500, unit: 'ml' },
      { name: 'Eieren', amount: 2, unit: 'stuk' },
      { name: 'Zout', amount: 1, unit: 'tl' },
    ],
    steps: 'Meng bloem, eieren en zout.\nVoeg de melk beetje bij beetje toe.\nBak dunne
pannenkoeken in een hete pan.',
  },
  {
    id: 2,
    title: 'Tomatensoep',
    description: 'Klassieke soep met verse tomaten.',
    category: 'lunch',
    servings: 4,
    minutes: 40,
    ingredients: [
      { name: 'Tomaten', amount: 800, unit: 'g' },
      { name: 'Ajuin', amount: 1, unit: 'stuk' },
      { name: 'Bouillon', amount: 500, unit: 'ml' },
      { name: 'Olijfolie', amount: 2, unit: 'el' },
    ],
    steps: 'Snijd ajuin en tomaten.\nBak de ajuin in olijfolie.\nVoeg tomaten en bouillon toe en
laat 25 minuten sudderen.\nMix de soep glad.',
  },
  {
    id: 3,
    title: 'Spaghetti bolognese',
    description: 'Een pot saus voor de hele week.',
    category: 'diner',
    servings: 4,
    minutes: 60,
    ingredients: [
      { name: 'Spaghetti', amount: 400, unit: 'g' },
      { name: 'Gehakt', amount: 500, unit: 'g' },
      { name: 'Tomaten', amount: 400, unit: 'g' },
      { name: 'Ajuin', amount: 1, unit: 'stuk' },
      { name: 'Olijfolie', amount: 1, unit: 'el' },
    ],
    steps: 'Bak ajuin en gehakt in olijfolie.\nVoeg de tomaten toe en laat 40 minuten pruttelen.
\nKook de spaghetti en meng ze met de saus.',
  },
  {
    id: 4,
    title: 'Kip curry',
    description: 'Romige curry met rijst.',
    category: 'diner',
    servings: 4,
    minutes: 45,
    ingredients: [
      { name: 'Kipfilet', amount: 500, unit: 'g' },
      { name: 'Rijst', amount: 300, unit: 'g' },
      { name: 'Kokosmelk', amount: 400, unit: 'ml' },
      { name: 'Ajuin', amount: 1, unit: 'stuk' },
      { name: 'Currypasta', amount: 2, unit: 'el' },
    ],
    steps: 'Snijd de kip in stukjes en bak ze aan.\nVoeg ajuin en currypasta toe.\nGiet de
kokosmelk erbij en laat 20 minuten sudderen.\nServeer met gekookte rijst.',
  },
  {
    id: 5,
    title: 'Griekse salade',
    description: 'Fris en klaar in een kwartier.',
    category: 'lunch',
    servings: 2,
    minutes: 15,
    ingredients: [
```

```
      { name: 'Tomaten', amount: 300, unit: 'g' },
      { name: 'Komkommer', amount: 1, unit: 'stuk' },
      { name: 'Feta', amount: 150, unit: 'g' },
      { name: 'Olijfolie', amount: 2, unit: 'el' },
    ],
    steps: 'Snijd tomaten en komkommer in blokjes.\nVerkruimel de feta erover.\nBesprenkel met
olijfolie.',
  },
  {
    id: 6,
    title: 'Chocolademousse',
    description: 'Luchtig dessert met pure chocolade.',
    category: 'dessert',
    servings: 6,
    minutes: 20,
    ingredients: [
      { name: 'Pure chocolade', amount: 200, unit: 'g' },
      { name: 'Eieren', amount: 4, unit: 'stuk' },
      { name: 'Slagroom', amount: 200, unit: 'ml' },
    ],
    steps:
'Smelt de chocolade.\nKlop de eiwitten stijf en de room op.\nHeb alles voorzichtig onder de
chocolade.\nLaat 3 uur opstijven in de koelkast.',
  },
];

// De voorbeeldrecepten zonder id, klaar om in een database te zetten.
export function sampleRecipes(): NewRecipe[] {
  return SAMPLE_RECIPES.map(({ id: _id, ...recipe }) => recipe);
}
```

Toelichting

- `as const` en `(typeof CATEGORIES)[number]` maken van een lijst een **union type**: `'ontbijt' | 'lunch' | 'diner' | 'dessert'`. Zo gebruik je één bron voor zowel de lijst (voor keuzelijsten) als het type.
- `interface` beschrijft de vorm van een object. `Omit<Recipe, 'id'>` is `Recipe` zonder `id`.
- `scaleAmount` rekent een hoeveelheid om: 250 g voor 4 personen wordt 125 g voor 2. We ronden af op 2 decimalen.
- Deze functies zijn **puur** (geen Angular, geen bijwerkingen). Daardoor zijn ze later heel makkelijk te testen (Deel 11).
- `steps` bewaren we als tekst waarin elke regel een stap is.

MODULE 6: Een herbruikbaar component met input() en output()

Doel van deze module

Na deze module kun je:

- een component genereren met de CLI,
- gegevens doorgeven met `input()`,
- een gebeurtenis naar boven sturen met `output()`,
- de nieuwe **control flow** in templates gebruiken: `@let`, `@switch`.

Stap 1: Genereren met de CLI

```
npx ng generate component recipes/recipe-card --skip-tests --style=none
```

Toelichting

- `generate component` (kort: `g c`) maakt `recipe-card.ts` en `recipe-card.html` in de map `src/app/recipes/recipe-card/`.
- `--skip-tests` slaat het testbestand over (we schrijven tests in Deel 11), `--style=none` maakt geen CSS-bestand (Tailwind regelt de stijl).
- De selector wordt automatisch `app-recipe-card`: de prefix `app` komt uit `angular.json`.

Stap 2: recipe-card.ts

Bestand: src/app/recipes/recipe-card/recipe-card.ts

```
import { Component, input, output } from '@angular/core';
import { Recipe } from '../recipe';

@Component({
  selector: 'app-recipe-card',
  templateUrl: './recipe-card.html',
})
export class RecipeCard {
  readonly recipe = input.required<Recipe>();
  readonly open = output<number>();
}
```

Stap 3: recipe-card.html

Bestand: src/app/recipes/recipe-card/recipe-card.html

```
@let r = recipe();

<article class="flex h-full flex-col rounded-xl border border-stone-200 bg-white p-4 shadow-sm">
  <div class="flex items-start justify-between gap-3">
    <h3 class="font-semibold text-stone-900">{{ r.title }}</h3>

    @switch (r.category) {
      @case ('ontbijt') {
        <span class="rounded-full bg-amber-100 px-2.5 py-1 text-xs font-medium text-amber-800">Ontb
ijt</span>
      }
      @case ('lunch') {
        <span class="rounded-full bg-sky-100 px-2.5 py-1 text-xs font-medium text-sky-800">Lunch</s
pan>
      }
      @case ('diner') {
        <span class="rounded-full bg-orange-100 px-2.5 py-1 text-xs font-medium text-orange-800">Di
ner</span>
      }
      @default {
        <span class="rounded-full bg-pink-100 px-2.5 py-1 text-xs font-medium text-pink-800">Desser
t</span>
      }
    }
  </div>

  <p class="mt-1 text-sm text-stone-600">{{ r.description }}</p>

  <p class="mt-3 text-sm text-stone-500">
    {{ r.minutes }} min &middot; {{ r.servings }} personen &middot; {{ r.ingredients.length }}
```

```
ingrediënten
  </p>

  <button
    type="button"
    (click)="open.emit(r.id)"
    class="mt-4 rounded-lg border border-stone-300 px-3 py-2 text-sm font-medium text-stone-700
hover:bg-stone-100"
  >
    Bekijk recept
  </button>
</article>
```

Toelichting: input() en output()

- `input.required<Recipe>()` declareert een **verplichte invoer**. Gebruik je het component zonder `[recipe]`, dan geeft Angular een compilatiefout. Voor een optionele invoer schrijf je `input('beginwaarde')`.
- Een `input` is zelf een **signal**: je leest hem met `recipe()`. Verandert de ouder de waarde, dan werkt de template vanzelf bij.
- `output<number>()` declareert een **gebeurtenis** die een `number` meegeeft. Het component stuurt hem met `open.emit(r.id)`. De ouder luistert met `(open)="..."`.

Toelichting: template-syntax

- `@let r = recipe();` maakt een lokale variabele in de template, zodat je niet telkens `recipe()` hoeft te schrijven.
- `@switch`/`@case`/`@default` is de template-versie van `switch`. Er komt maar één tak in de pagina.
- `·` is een HTML-teken voor een middenpunt. De klassen staan rechtstreeks in de template.

> **Vergelijking met Vue/React:** `input()` is `defineProps` (Vue) of props (React); `output()` is `defineEmits` (Vue) of een callback-prop (React). Het grote verschil met React: in Angular geef je **geen functies** door als props, je luistert naar een gebeurtenis met `(naam)="..."`.

MODULE 7: Content projection met ng-content

Doel van deze module

Een container-component maken waarin je zelf inhoud plaatst. Dit is Angular's versie van `<slot>` (Vue) en `children` (React).

```
npx ng generate component shared/panel --skip-tests --style=none
```

Bestand: src/app/shared/panel/panel.ts

```
import { Component, input } from '@angular/core';

@Component({
  selector: 'app-panel',
  templateUrl: './panel.html',
})
export class Panel {
```

```
  readonly title = input.required<string>();
}
```

Bestand: src/app/shared/panel/panel.html

```
<section class="rounded-xl border border-stone-200 bg-white p-4 shadow-sm">
  <div class="mb-3 flex items-center justify-between gap-3">
    <h2 class="text-lg font-semibold text-stone-800">{{ title() }}</h2>
    <ng-content select="[panel-actions]" />
  </div>

  <ng-content />
</section>
```

Toelichting

- `<ng-content />` is de **uitsparing**: alles wat je tussen `<app-panel>` en `</app-panel>` schrijft, verschijnt hier.
- `<ng-content select="[panel-actions]" />` is een **benoemde** uitsparing. Alleen elementen met het attribuut `panel-actions` komen hier terecht; al het andere komt in de standaard `<ng-content />`.
- Een voorbeeld van het gebruik zie je in de volgende module.

MODULE 8: Een lijst met filter, @for en computed()

Doel van deze module

Na deze module kun je:

- een lijst tonen met `@for` (inclusief `track` en `@empty`),
- een afgeleide waarde berekenen met `computed()`,
- een filter in een signal bewaren.

Stap 1: app.ts

Bestand: src/app/app.ts

```
import { Component, computed, signal } from '@angular/core';
import { Panel } from './shared/panel/panel';
import { RecipeCard } from './recipes/recipe-card/recipe-card';
import { CATEGORIES, Category, Recipe, SAMPLE_RECIPES } from './recipes/recipe';

@Component({
  selector: 'app-root',
  imports: [Panel, RecipeCard],
  templateUrl: './app.html',
})
export class App {
  protected readonly categories = CATEGORIES;
  protected readonly recipes = signal<Recipe[]>(SAMPLE_RECIPES);
  protected readonly category = signal<Category | 'alle'>('alle');

  protected readonly visible = computed(() => {
    const selected = this.category();
    const all = this.recipes();
    return selected === 'alle' ? all : all.filter((recipe) => recipe.category === selected);
  });
```

```
  protected readonly chip = 'rounded-full border border-stone-300 px-3 py-1 text-sm capitalize
text-stone-700 hover:bg-stone-100';
  protected readonly chipActive = 'rounded-full border border-orange-600 bg-orange-600 px-3 py-1
text-sm capitalize text-white';

  protected select(category: Category | 'alle'): void {
    this.category.set(category);
  }

  protected open(id: number): void {
    console.log('Open recept', id);
  }
}
```

Stap 2: app.html

Bestand: src/app/app.html

```
<div class="min-h-screen bg-stone-50">
  <header class="border-b border-stone-200 bg-white">
    <div class="mx-auto max-w-5xl px-6 py-4">
      <h1 class="text-2xl font-bold text-orange-700">Weekmenu</h1>
      <p class="text-sm text-stone-500">Plan je recepten en krijg vanzelf een boodschappenlijst.</
p>
    </div>
  </header>

  <main class="mx-auto max-w-5xl space-y-6 p-6">
    <app-panel title="Filter">
      <span panel-actions class="text-sm text-stone-500">
        {{ visible().length }} van {{ recipes().length }} recepten
      </span>

      <div class="flex flex-wrap gap-2">
        <button type="button" (click)="select('alle')" [class]="category() === 'alle' ? chipActi
ve : chip">
          Alle
        </button>

        @for (item of categories; track item) {
          <button type="button" (click)="select(item)" [class]="category() === item ?
chipActive : chip">
            {{ item }}
          </button>
        }
      </div>
    </app-panel>

    <div class="grid gap-4 sm:grid-cols-2 lg:grid-cols-3">
      @for (recipe of visible(); track recipe.id) {
        <app-recipe-card [recipe]="recipe" (open)="open($event)" />
      } @empty {
        <p class="col-span-full rounded-xl border border-dashed border-stone-300 p-8 text-center
text-stone-500">
          Geen recepten gevonden.
        </p>
      }
    </div>
  </main>
</div>
```

Toelichting: de klasse

- `imports: [Panel, RecipeCard]`: de template mag alleen componenten gebruiken die hier staan. Vergeet je er één, dan geeft de compiler een duidelijke fout.

- `computed(() => ...)` maakt een **afgeleide signal**. Hij rekent opnieuw uit zodra `category()` of `recipes()` verandert, en onthoudt anders het vorige resultaat. Je kunt een computed niet zelf zetten: hij is alleen-lezen.
- `Category | 'alle'` is een union: het filter is een categorie of "alles".

Toelichting: de template

- `@for (item of lijst; track item.id) { ... }` herhaalt een blok. **`track` is verplicht**: net als `key` in Vue en React vertelt het Angular hoe items herkend worden, zodat bij een wijziging alleen de gewijzigde items bijgewerkt worden. Gebruik een stabiel id; voor een lijst teksten kun je het item zelf gebruiken.
- `@empty { ... }` toont inhoud als de lijst leeg is.
- `[class]="voorwaarde ? a : b"` bindt een volledige klassenreeks.
- `(open)="open($event)"`: `$event` bevat de waarde die het component met `emit` meestuurde, hier het id.
- `<span panel-actions>` komt dankzij het attribuut in de benoemde uitsparing van de `Panel` (rechtsboven). De knoppen komen in de standaard uitsparing.

> **Vergelijking met Vue/React:** `computed()` is `computed()` in Vue. In React reken je gewoon tijdens het renderen uit. `@for` met `track` is `v-for` met `:key` en `map` met `key`. `@if`/`@else` vervangen `v-if` en voorwaardelijk renderen.

Oefeningen

1. Voeg een zoekveld toe dat op titel filtert. Gebruik een tweede signal `query` en pas `visible` aan. Een invoerveld koppel je aan een signal met `[value]="query()"` en `(input)="query.set($any($event.target).value)"`. (In Deel 5 zie je een nettere manier.)
2. Toon in `RecipeCard` een label "Snel" als `r.minutes <= 20`, met `@if (r.minutes <= 20) { ... }`.
3. Voeg een vijfde categorie toe aan `CATEGORIES`. De `@switch` valt terug op `@default`; pas hem aan zodat er een extra `@case` bij komt.

Deel 3: Services en dependency injection

MODULE 9: Wat is een service en wat is dependency injection?

In Deel 2 staan de recepten in de `App`-klasse. Zodra meerdere pagina's dezelfde recepten nodig hebben (lijst, detail, planner), is dat onhandig. In Angular zet je gedeelde data en logica in een **service**: een gewone klasse die Angular **voor jou aanmaakt en doorgeeft**.

Het mechanisme heet **dependency injection (DI)**:

1. Je markeert een klasse als service met `@Service()`.
2. Een component vraagt erom met `inject(NaamVanDeService)`.
3. Angular maakt de service **één keer** aan en geeft **dezelfde instantie** aan iedereen die erom vraagt.

Voordelen: je hoeft nergens `new RecipeStore()` te schrijven, gedeelde data staat op één plek, en in tests kun je een service eenvoudig vervangen door een nepversie.

> **Vergelijking met Vue/React:** een service met signals is ongeveer een Pinia-store (Vue) of een Context met reducer (React). Het is het standaard Angular-patroon voor gedeelde state.

MODULE 10: De RecipeStore

Stap 1: Genereren

```
npx ng generate service recipes/recipe-store --skip-tests
```

Stap 2: recipe-store.ts

Bestand: src/app/recipes/recipe-store.ts

```
import { computed, Service, signal } from '@angular/core';
import { NewRecipe, Recipe, SAMPLE_RECIPES } from './recipe';

@Service()
export class RecipeStore {
  private readonly state = signal<Recipe[]>(SAMPLE_RECIPES);

  readonly recipes = this.state.asReadonly();
  readonly count = computed(() => this.state().length);

  byId(id: number): Recipe | undefined {
    return this.state().find((recipe) => recipe.id === id);
  }

  add(recipe: NewRecipe): Recipe {
    const id = this.state().reduce((max, item) => Math.max(max, item.id), 0) + 1;
    const created: Recipe = { ...recipe, id };
    this.state.update((list) => [...list, created]);
```

```
    return created;
  }

  update(id: number, changes: NewRecipe): void {
    this.state.update((list) => list.map((item) => (item.id === id ? { ...changes, id } : item)));
  }

  remove(id: number): void {
    this.state.update((list) => list.filter((item) => item.id !== id));
  }
}
```

Toelichting

- `@Service()` is de decorator voor een service die in de hele applicatie beschikbaar is (één gedeelde instantie). In oudere code zie je `@Injectable({ providedIn: 'root' })`; het doet hetzelfde (Bijlage A).
- `state` is **private** en schrijfbaar. `recipes` is een **alleen-lezen** versie (`asReadonly()`): componenten kunnen de lijst lezen, maar alleen de service mag hem wijzigen. Zo weet je altijd waar wijzigingen vandaan komen.
- De methoden gebruiken `update` met **nieuwe arrays** (`[...list, nieuw]`, `map`, `filter`). Signals herkennen een wijziging doordat de waarde verandert; een bestaande array aanpassen met `push` merkt Angular niet.

MODULE 11: De service gebruiken met inject()

Pas `app.ts` aan: de recepten komen nu uit de store.

Bestand: src/app/app.ts

```
import { Component, computed, inject, signal } from '@angular/core';
import { Panel } from './shared/panel/panel';
import { RecipeCard } from './recipes/recipe-card/recipe-card';
import { CATEGORIES, Category } from './recipes/recipe';
import { RecipeStore } from './recipes/recipe-store';

@Component({
  selector: 'app-root',
  imports: [Panel, RecipeCard],
  templateUrl: './app.html',
})
export class App {
  private readonly store = inject(RecipeStore);

  protected readonly categories = CATEGORIES;
  protected readonly recipes = this.store.recipes;
  protected readonly category = signal<Category | 'alle'>('alle');

  protected readonly visible = computed(() => {
    const selected = this.category();
    const all = this.recipes();
    return selected === 'alle' ? all : all.filter((recipe) => recipe.category === selected);
  });

  protected readonly chip = 'rounded-full border border-stone-300 px-3 py-1 text-sm capitalize
text-stone-700 hover:bg-stone-100';
  protected readonly chipActive = 'rounded-full border border-orange-600 bg-orange-600 px-3 py-1
text-sm capitalize text-white';

  protected select(category: Category | 'alle'): void {
    this.category.set(category);
```

```
  }

  protected open(id: number): void {
    console.log('Open recept', id);
  }
}
```

Toelichting

- `inject(RecipeStore)` vraagt Angular om de gedeelde instantie. Je schrijft dit als **veldinitialisatie** bovenaan de klasse.
- De template (`app.html`) blijft ongewijzigd: `recipes()` is nu de alleen-lezen signal van de store.
- In oudere code zie je constructor-injectie: `constructor(private store: RecipeStore) {}`. Dat werkt nog steeds, maar `inject()` is korter en werkt ook in functies (guards, interceptors).

Hoe werkt het bereik van een service?

Registratie	Bereik
`@Service()`	één instantie voor de hele app (meestal wat je wilt)
`providers: [X]` in een component	een **eigen** instantie voor dat component en zijn kinderen
`providers: [...]` in `app.config.ts`	de hele app, handig voor functies zoals `provideRouter`

Angular zoekt een service in een **boom van injectors**, van het component omhoog tot de root. Dit is de reden dat je een service vervangen kunt door er lager in de boom een andere voor te leggen (nuttig bij tests).

Oefeningen

1. Voeg een computed `byCategory` toe aan de store die een object teruggeeft met per categorie het aantal recepten.
2. Voeg een knop "Voorbeeldrecept toevoegen" toe aan `App` die `store.add(...)` aanroept met een eigen `NewRecipe`. Het nieuwe recept verschijnt meteen in de lijst en de teller, zonder dat je iets extra hoeft te doen.

Deel 4: Routing

MODULE 12: Pagina's, routes en het app-shell

Doel van deze module

Na deze module kun je:

- een route koppelen aan een component,
- een gedeeld **shell** (header en navigatie) maken met `<router-outlet />`,
- links maken met `routerLink`.

Hoe routing werkt

In Angular definieer je routes **in code**, in `app.routes.ts`: een lijst met objecten `{ path, component }`. De router kijkt naar de URL, kiest de passende route en tekent het bijbehorende component op de plek van `<router-outlet />`.

> **Vergelijking met Vue/React:** Nuxt en Next.js bepalen routes aan de hand van **mappen**. Angular (net als Vue Router en React Router) gebruikt een **lijst in code**. Je hebt dus meer controle, maar je moet elke route zelf registreren.

Stap 1: Drie pagina's genereren

```
npx ng generate component recipes/recipe-list --skip-tests --style=none
npx ng generate component recipes/recipe-detail --skip-tests --style=none
npx ng generate component shared/not-found --skip-tests --style=none
```

Stap 2: De routes

Bestand: src/app/app.routes.ts

```
import { Routes } from '@angular/router';

export const routes: Routes = [
  { path: '', pathMatch: 'full', redirectTo: 'recepten' },
  {
    path: 'recepten',
    title: 'Recepten',
    loadComponent: () => import('./recipes/recipe-list/recipe-list').then((m) => m.RecipeList),
  },
  {
    path: 'recepten/:id',
    title: 'Recept',
    loadComponent: () => import('./recipes/recipe-detail/recipe-detail').then((m) => m.RecipeDetail
),
  },
  {
    path: '**',
```

```
    title: 'Niet gevonden',
    loadComponent: () => import('./shared/not-found/not-found').then((m) => m.NotFound),
  },
];
```

Toelichting

- `path: ''` met `pathMatch: 'full'` en `redirectTo` stuurt de lege URL (`/`) door naar `/recepten`.
- `path: 'recepten/:id'`: het deel met de dubbele punt is een **parameter**. De URL `/recepten/3` heeft `id = '3'` (altijd een tekst).
- `title` zet de titel van het browsertabblad voor die route. Daar hoef je geen eigen code voor te schrijven.
- **`loadComponent: () => import(...)` is lazy loading.** De code van een pagina wordt pas gedownload als iemand ernaartoe navigeert. Zo blijft je eerste download klein. Bij het bouwen maakt Angular er automatisch aparte bestanden van.
- `path: '**'` is de **wildcard**: alles wat nergens anders past. Hij moet **onderaan** staan, want de router neemt de **eerste** route die past.

Stap 3: De router configureren

Bestand: src/app/app.config.ts

```
import { ApplicationConfig, provideBrowserGlobalErrorListeners } from '@angular/core';
import { provideRouter, withComponentInputBinding } from '@angular/router';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideRouter(routes, withComponentInputBinding()),
  ],
};
```

Toelichting

- `withComponentInputBinding()` is een router-optie die **routeparameters automatisch aan `input()`s koppelt**. Heet de parameter `id` en heeft het component `input<string>()` met de naam `id`, dan krijgt het component de waarde vanzelf. Zie Module 13.

Stap 4: Het shell

Bestand: src/app/app.ts

```
import { Component } from '@angular/core';
import { RouterLink, RouterLinkActive, RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet, RouterLink, RouterLinkActive],
  templateUrl: './app.html',
})
export class App {
  protected readonly link = 'rounded-lg px-3 py-2 text-sm font-medium text-stone-600 hover:bg-
stone-100';
}
```

Bestand: src/app/app.html

```
<div class="min-h-screen bg-stone-50">
  <header class="border-b border-stone-200 bg-white">
    <div class="mx-auto flex max-w-5xl flex-wrap items-center justify-between gap-3 px-6 py-4">
      <a routerLink="/recepten" class="text-xl font-bold text-orange-700">Weekmenu</a>

      <nav class="flex gap-1">
        <a
          routerLink="/recepten"
          routerLinkActive="bg-orange-100 text-orange-800"
          [routerLinkActiveOptions]="{ exact: true }"
          [class]="link"
        >
          Recepten
        </a>
      </nav>
    </div>
  </header>

  <main class="mx-auto max-w-5xl p-6">
    <router-outlet />
  </main>
</div>
```

Toelichting

- `<router-outlet />` is de plek waar de component van de actieve route verschijnt. De rest (de header) blijft staan bij navigatie.
- `routerLink="/recepten"` vervangt `href`: de pagina wordt niet opnieuw geladen. Gebruik voor interne links altijd `routerLink`. Voor een dynamisch pad schrijf je `[routerLink]="['/recepten', recipe.id]"`.
- `routerLinkActive="klassen"` voegt die klassen toe als de link bij de huidige URL hoort. Met `{ exact: true }` geldt dat alleen als het pad **precies** gelijk is.
- Je ziet een lege pagina onder de header: de pagina-componenten zijn nog leeg. Die vullen we nu.

MODULE 13: De lijstpagina, de detailpagina en routeparameters

Stap 1: De lijstpagina

Bestand: src/app/recipes/recipe-list/recipe-list.ts

```
import { Component, computed, inject, signal } from '@angular/core';
import { Router } from '@angular/router';
import { Panel } from '../../shared/panel/panel';
import { CATEGORIES, Category } from '../recipe';
import { RecipeCard } from '../recipe-card/recipe-card';
import { RecipeStore } from '../recipe-store';

@Component({
  selector: 'app-recipe-list',
  imports: [Panel, RecipeCard],
  templateUrl: './recipe-list.html',
})
export class RecipeList {
  private readonly store = inject(RecipeStore);
  private readonly router = inject(Router);

  protected readonly categories = CATEGORIES;
  protected readonly recipes = this.store.recipes;
  protected readonly category = signal<Category | 'alle'>('alle');
```

```
  protected readonly visible = computed(() => {
    const selected = this.category();
    const all = this.recipes();
    return selected === 'alle' ? all : all.filter((recipe) => recipe.category === selected);
  });

  protected readonly chip = 'rounded-full border border-stone-300 px-3 py-1 text-sm capitalize
text-stone-700 hover:bg-stone-100';
  protected readonly chipActive = 'rounded-full border border-orange-600 bg-orange-600 px-3 py-1
text-sm capitalize text-white';

  protected select(category: Category | 'alle'): void {
    this.category.set(category);
  }

  protected open(id: number): void {
    this.router.navigate(['/recepten', id]);
  }
}
```

Bestand: src/app/recipes/recipe-list/recipe-list.html

```
<div class="space-y-6">
  <app-panel title="Filter">
    <span panel-actions class="text-sm text-stone-500">
      {{ visible().length }} van {{ recipes().length }} recepten
    </span>

    <div class="flex flex-wrap gap-2">
      <button type="button" (click)="select('alle')" [class]="category() === 'alle' ?
chipActive : chip">
        Alle
      </button>

      @for (item of categories; track item) {
        <button type="button" (click)="select(item)" [class]="category() === item ?
chipActive : chip">
          {{ item }}
        </button>
      }
    </div>
  </app-panel>

  <div class="grid gap-4 sm:grid-cols-2 lg:grid-cols-3">
    @for (recipe of visible(); track recipe.id) {
      <app-recipe-card [recipe]="recipe" (open)="open($event)" />
    } @empty {
      <p class="col-span-full rounded-xl border border-dashed border-stone-300 p-8 text-center
text-stone-500">
        Geen recepten gevonden.
      </p>
    }
  </div>
</div>
```

Toelichting

- Dit is de inhoud van `App` uit Deel 3, nu verhuisd naar een eigen pagina.
- `this.router.navigate(['/recepten', id])` navigeert **vanuit code**. De array wordt tot het pad `/recepten/3` samengevoegd. Gebruik `navigate` na een actie (bijvoorbeeld opslaan) en `routerLink` voor gewone links.

Stap 2: De detailpagina

Bestand: src/app/recipes/recipe-detail/recipe-detail.ts

```
import { Component, computed, inject, input, linkedSignal } from '@angular/core';
import { Router, RouterLink } from '@angular/router';
import { formatAmount, scaleAmount } from '../recipe';
import { RecipeStore } from '../recipe-store';

@Component({
  selector: 'app-recipe-detail',
  imports: [RouterLink],
  templateUrl: './recipe-detail.html',
})
export class RecipeDetail {
  private readonly store = inject(RecipeStore);
  private readonly router = inject(Router);

  // Komt uit de URL: /recepten/:id
  readonly id = input.required<string>();

  protected readonly recipe = computed(() => this.store.byId(Number(this.id())));

  // Begint bij het aantal personen van het recept en wordt opnieuw ingesteld als het recept
verandert.
  protected readonly servings = linkedSignal(() => this.recipe()?.servings ?? 1);

  protected readonly ingredients = computed(() => {
    const recipe = this.recipe();
    if (!recipe) return [];

    return recipe.ingredients.map((ingredient) => ({
      name: ingredient.name,
      text: formatAmount(scaleAmount(ingredient.amount, recipe.servings, this.servings()), ingredie
nt.unit),
    }));
  });

  protected readonly steps = computed(
    () => this.recipe()?.steps.split('\n').filter((line) => line.trim() !== '') ?? [],
  );

  protected change(delta: number): void {
    this.servings.update((value) => Math.min(24, Math.max(1, value + delta)));
  }

  protected remove(id: number): void {
    this.store.remove(id);
    this.router.navigateByUrl('/recepten');
  }
}
```

Bestand: src/app/recipes/recipe-detail/recipe-detail.html

```
@if (recipe(); as r) {
  <div class="space-y-6">
    <a routerLink="/recepten" class="text-sm text-orange-700 hover:underline">Terug naar alle
recepten</a>

    <article class="rounded-xl border border-stone-200 bg-white p-6 shadow-sm">
      <div class="flex flex-wrap items-start justify-between gap-3">
        <div>
          <h1 class="text-2xl font-bold">{{ r.title }}</h1>
          <p class="text-stone-600">{{ r.description }}</p>
        </div>
        <span class="text-sm text-stone-500">{{ r.minutes }} minuten</span>
      </div>

      <div class="mt-6 flex items-center gap-3">
        <span class="text-sm font-medium text-stone-700">Aantal personen</span>
        <button type="button" (click)="change(-1)" class="h-8 w-8 rounded-lg border border-
```

```
stone-300 hover:bg-stone-100">-</button>
        <span class="w-6 text-center font-semibold">{{ servings() }}</span>
        <button type="button" (click)="change(1)" class="h-8 w-8 rounded-lg border border-
stone-300 hover:bg-stone-100">+</button>
      </div>

      <h2 class="mt-6 text-lg font-semibold">Ingrediënten</h2>
      <ul class="mt-2">
        @for (item of ingredients(); track $index) {
          <li class="flex justify-between border-b border-stone-100 py-1">
            <span>{{ item.name }}</span>
            <span class="text-stone-600">{{ item.text }}</span>
          </li>
        }
      </ul>

      <h2 class="mt-6 text-lg font-semibold">Bereiding</h2>
      <ol class="mt-2 list-decimal space-y-1 pl-5">
        @for (step of steps(); track $index) {
          <li>{{ step }}</li>
        }
      </ol>
    </article>

    <button
      type="button"
      (click)="remove(r.id)"
      class="text-sm text-stone-500 underline hover:text-red-700"
    >
      Recept verwijderen
    </button>
  </div>
} @else {
  <p class="text-stone-600">
    Dit recept bestaat niet (meer).
    <a routerLink="/recepten" class="text-orange-700 hover:underline">Terug naar alle recepten</a>
  </p>
}
```

Toelichting: parameters in de URL

- `readonly id = input.required<string>();`: door `withComponentInputBinding()` (Module 12) krijgt dit component de waarde van `:id` uit de URL **als gewone input**. Je hoeft `ActivatedRoute` en `subscribe` niet te gebruiken. In oudere code zie je `this.route.snapshot.paramMap.get('id')`; dat werkt nog steeds (Bijlage A).
- `Number(this.id())` zet de tekst om naar een getal. Een URL-waarde is altijd tekst en kan onzin zijn (`/recepten/abc`): `byId` geeft dan `undefined` en de template toont de tak `@else`.
- `@if (recipe(); as r) { ... } @else { ... }` bewaart de uitkomst in `r` (en controleert meteen of die bestaat).

Toelichting: linkedSignal

- Navigeer je van `/recepten/1` naar `/recepten/2`, dan **hergebruikt Angular hetzelfde component**: alleen de input verandert. Een gewone `signal(…)` zou dan het oude aantal personen blijven tonen.
- `linkedSignal(() => ...)` is een signal die een **beginwaarde berekent uit andere signals** en je daarna zelf kunt aanpassen met `set`/`update`. Verandert het recept, dan wordt hij **opnieuw** op de beginwaarde gezet. Perfect voor "een waarde die de gebruiker mag wijzigen, maar die reset als de bron verandert".

Stap 3: De 404-pagina

Bestand: src/app/shared/not-found/not-found.ts

```
import { Component } from '@angular/core';
import { RouterLink } from '@angular/router';

@Component({
  selector: 'app-not-found',
  imports: [RouterLink],
  template: `
    <div class="py-16 text-center">
      <h1 class="text-3xl font-bold text-stone-900">Niet gevonden</h1>
      <p class="mt-2 text-stone-600">Deze pagina bestaat niet.</p>
      <a routerLink="/recepten" class="mt-6 inline-block text-orange-700 hover:underline">Naar de
recepten</a>
    </div>
  `,
})
export class NotFound {}
```

Toelichting: een klein component kan zijn template direct in de klasse (`template`) hebben, in plaats van in een apart bestand. De CLI heeft `templateUrl` gegenereerd; vervang het door `template` zoals hierboven.

Test het: klik op "Bekijk recept", wijzig het aantal personen en kijk hoe de hoeveelheden meeschalen. Ga naar `/recepten/999` en naar `/bestaat-niet`.

Deel 5: Formulieren met Signal Forms

MODULE 14: Welke formulieren heeft Angular?

Angular kent drie manieren om formulieren te bouwen:

Manier	Wanneer?
Signal Forms (`@angular/forms/signals`)	De nieuwste, **stabiele** aanpak sinds Angular 22. Het formuliermodel is een signal; gebruik dit voor nieuwe code.
Reactive Forms (`FormGroup`, `FormControl`)	De klassieke aanpak met objecten en RxJS. Je ziet dit in **bijna alle bestaande projecten**. Zie Deel 6 en 7.
Template-driven forms (`ngModel`)	Oud en eenvoudig; minder geschikt voor complexe formulieren.

In dit deel bouw je het receptformulier met **Signal Forms**. In Deel 6 en 7 gebruik je daarnaast Reactive Forms, zodat je beide kent.

Het idee achter Signal Forms

1. Je bewaart de data van het formulier in een **gewone signal** (het *model*).
2. `form(model, schema)` maakt daar een **formulierboom** van, met per veld een status (waarde, `touched`, `valid`, `errors`).
3. Je koppelt invoervelden met de directive **`[formField]`**.
4. Je beschrijft regels (validatie) in een **schema**-functie.

De data en het formulier zijn dus **dezelfde bron**. Er is geen aparte kopie die je moet synchroniseren.

MODULE 15: Een herbruikbare foutmelding

Eerst een klein component dat foutmeldingen toont. We gebruiken het bij elk veld.

```
npx ng generate component shared/form-errors --skip-tests --style=none
```

Bestand: src/app/shared/form-errors/form-errors.ts

```ts
import { Component, input } from '@angular/core';

export interface FieldError {
  kind: string;
  message?: string;
}

@Component({
  selector: 'app-form-errors',
  template: `
    @for (error of errors(); track error.kind) {
      <p class="mt-1 text-sm text-red-700">{{ error.message ?? error.kind }}</p>
    }
```

```
  `,
})
export class FormErrors {
  readonly errors = input.required<readonly FieldError[]>();
}
```

Het bestand `form-errors.html` dat de CLI maakte, mag je verwijderen:

```
del src\app\shared\form-errors\form-errors.html
```

MODULE 16: Het receptformulier

Doel van deze module

Een formulier voor **nieuw** en **bestaand** recept, met een dynamische lijst ingrediënten en validatie.

Stap 1: Genereren

```
npx ng generate component recipes/recipe-form --skip-tests --style=none
```

Stap 2: recipe-form.ts

Bestand: src/app/recipes/recipe-form/recipe-form.ts

```
import { Component, computed, inject, input, linkedSignal, signal } from '@angular/core';
import { applyEach, form, FormField, max, min, minLength, required, submit } from '@angular/forms/
signals';
import { Router, RouterLink } from '@angular/router';
import { FieldError, FormErrors } from '../../shared/form-errors/form-errors';
import { CATEGORIES, Category, Ingredient, NewRecipe, Recipe, UNITS } from '../recipe';
import { RecipeStore } from '../recipe-store';

interface RecipeModel {
  title: string;
  description: string;
  category: Category;
  servings: number;
  minutes: number;
  steps: string;
  ingredients: Ingredient[];
}

function emptyIngredient(): Ingredient {
  return { name: '', amount: 1, unit: 'g' };
}

function emptyModel(): RecipeModel {
  return {
    title: '',
    description: '',
    category: 'diner',
    servings: 4,
    minutes: 30,
    steps: '',
    ingredients: [emptyIngredient()],
  };
}

function toModel(recipe: Recipe): RecipeModel {
  return {
```

```
    title: recipe.title,
    description: recipe.description,
    category: recipe.category,
    servings: recipe.servings,
    minutes: recipe.minutes,
    steps: recipe.steps,
    ingredients: recipe.ingredients.map((ingredient) => ({ ...ingredient })),
  };
}

@Component({
  selector: 'app-recipe-form',
  imports: [FormField, FormErrors, RouterLink],
  templateUrl: './recipe-form.html',
})
export class RecipeForm {
  private readonly store = inject(RecipeStore);
  private readonly router = inject(Router);

  // Alleen aanwezig op de route /recepten/:id/bewerken
  readonly id = input<string>();

  protected readonly categories = CATEGORIES;
  protected readonly units = UNITS;

  protected readonly existing = computed(() => {
    const id = this.id();
    return id === undefined ? undefined : this.store.byId(Number(id));
  });

  // Het formuliermodel: leeg bij een nieuw recept, gevuld bij bewerken.
  protected readonly model = linkedSignal<RecipeModel>(() => {
    const recipe = this.existing();
    return recipe ? toModel(recipe) : emptyModel();
  });

  protected readonly recipeForm = form(this.model, (path) => {
    required(path.title, { message: 'Geef het recept een titel.' });
    minLength(path.title, 3, { message: 'De titel heeft minstens 3 tekens nodig.' });
    required(path.steps, { message: 'Beschrijf de bereiding.' });
    min(path.servings, 1, { message: 'Minstens 1 persoon.' });
    max(path.servings, 24, { message: 'Maximaal 24 personen.' });
    min(path.minutes, 1, { message: 'Minstens 1 minuut.' });
    max(path.minutes, 600, { message: 'Maximaal 600 minuten.' });
    minLength(path.ingredients, 1, { message: 'Voeg minstens één ingrediënt toe.' });

    applyEach(path.ingredients, (item) => {
      required(item.name, { message: 'Naam ontbreekt.' });
      min(item.amount, 0.01, { message: 'De hoeveelheid moet groter zijn dan 0.' });
    });
  });

  protected readonly saving = signal(false);
  protected readonly saveError = signal<string | null>(null);

  protected readonly input =
    'w-full rounded-lg border border-stone-300 bg-white px-3 py-2 text-sm focus:border-orange-500
focus:outline-none focus:ring-2 focus:ring-orange-200';
  protected readonly label = 'mb-1 block text-sm font-medium text-stone-700';

  // Toon fouten pas nadat het veld bezocht is.
  protected shown(state: { touched(): boolean; errors(): readonly FieldError[] }): readonly FieldEr
ror[] {
    return state.touched() ? state.errors() : [];
  }

  protected addIngredient(): void {
    this.model.update((current) => ({ ...current, ingredients: [...current.ingredients, emptyIngred
ient()] }));
  }

  protected removeIngredient(index: number): void {
    this.model.update((current) => ({
```

```
      ...current,
      ingredients: current.ingredients.filter((_, position) => position !== index),
    }));
  }

  protected async save(event: Event): Promise<void> {
    event.preventDefault();
    this.saveError.set(null);

    await submit(this.recipeForm, async () => {
      this.saving.set(true);

      try {
        const value: NewRecipe = { ...this.model() };
        const existing = this.existing();

        if (existing) {
          this.store.update(existing.id, value);
          await this.router.navigate(['/recepten', existing.id]);
        } else {
          const created = this.store.add(value);
          await this.router.navigate(['/recepten', created.id]);
        }
      } catch (error) {
        this.saveError.set(error instanceof Error ? error.message : 'Opslaan is mislukt.');
      } finally {
        this.saving.set(false);
      }
    });
  }
}
```

Stap 3: recipe-form.html

Bestand: src/app/recipes/recipe-form/recipe-form.html

```
<form (submit)="save($event)" novalidate class="space-y-6">
  <div class="flex flex-wrap items-center justify-between gap-3">
    <h1 class="text-2xl font-bold">{{ existing() ? 'Recept bewerken' : 'Nieuw recept' }}</h1>
    <a routerLink="/recepten" class="text-sm text-orange-700 hover:underline">Annuleren</a>
  </div>

  <div class="grid gap-4 rounded-xl border border-stone-200 bg-white p-6 shadow-sm sm:grid-cols-2">
    <div class="sm:col-span-2">
      <label for="title" [class]="label">Titel</label>
      <input id="title" type="text" [formField]="recipeForm.title" [class]="input" />
      <app-form-errors [errors]="shown(recipeForm.title())" />
    </div>

    <div class="sm:col-span-2">
      <label for="description" [class]="label">Korte beschrijving</label>
      <input id="description" type="text" [formField]="recipeForm.description" [class]="input" />
    </div>

    <div>
      <label for="category" [class]="label">Categorie</label>
      <select id="category" [formField]="recipeForm.category" [class]="input">
        @for (item of categories; track item) {
          <option [value]="item">{{ item }}</option>
        }
      </select>
    </div>

    <div>
      <label for="minutes" [class]="label">Bereidingstijd (minuten)</label>
      <input id="minutes" type="number" [formField]="recipeForm.minutes" [class]="input" />
      <app-form-errors [errors]="shown(recipeForm.minutes())" />
    </div>

    <div>
      <label for="servings" [class]="label">Aantal personen</label>
```

```
      <input id="servings" type="number" [formField]="recipeForm.servings" [class]="input" />
      <app-form-errors [errors]="shown(recipeForm.servings())" />
    </div>
  </div>

  <div class="rounded-xl border border-stone-200 bg-white p-6 shadow-sm">
    <div class="mb-3 flex items-center justify-between">
      <h2 class="text-lg font-semibold">Ingrediënten</h2>
      <button type="button" (click)="addIngredient()" class="rounded-lg border border-stone-300
px-3 py-1.5 text-sm hover:bg-stone-100">
        Ingrediënt toevoegen
      </button>
    </div>

    <app-form-errors [errors]="shown(recipeForm.ingredients())" />

    <div class="space-y-3">
      @for (ingredient of recipeForm.ingredients; track $index) {
        <div class="grid grid-cols-[1fr_6rem_6rem_auto] items-start gap-2">
          <div>
            <input type="text" placeholder="Naam" [formField]="ingredient.name" [class]="input" /
>
            <app-form-errors [errors]="shown(ingredient.name())" />
          </div>
          <div>
            <input type="number" step="any" [formField]="ingredient.amount" [class]="input" />
            <app-form-errors [errors]="shown(ingredient.amount())" />
          </div>
          <select [formField]="ingredient.unit" [class]="input">
            @for (unit of units; track unit) {
              <option [value]="unit">{{ unit }}</option>
            }
          </select>
          <button
            type="button"
            (click)="removeIngredient($index)"
            aria-label="Ingrediënt verwijderen"
            class="rounded-lg border border-stone-300 px-3 py-2 text-sm hover:bg-stone-100"
          >
            X
          </button>
        </div>
      }
    </div>
  </div>

  <div class="rounded-xl border border-stone-200 bg-white p-6 shadow-sm">
    <label for="steps" [class]="label">Bereiding (één stap per regel)</label>
    <textarea id="steps" rows="6" [formField]="recipeForm.steps" [class]="input"></textarea>
    <app-form-errors [errors]="shown(recipeForm.steps())" />
  </div>

  <div class="flex items-center gap-4">
    <button
      type="submit"
      [disabled]="saving()"
      class="rounded-lg bg-orange-600 px-5 py-2 font-medium text-white hover:bg-orange-700
disabled:opacity-60"
    >
      {{ saving() ? 'Bezig met opslaan...' : 'Opslaan' }}
    </button>
    @if (saveError(); as message) {
      <span role="alert" class="text-sm text-red-700">{{ message }}</span>
    }
    @if (recipeForm().invalid() && recipeForm().touched()) {
      <span class="text-sm text-red-700">Controleer de velden met een foutmelding.</span>
    }
  </div>
</form>
```

Toelichting: het model en het formulier

- `RecipeModel` beschrijft de data van het formulier.
- `linkedSignal<RecipeModel>(...)` maakt het model: leeg bij een nieuw recept, gevuld bij bewerken. Doordat het een linkedSignal is, wordt het opnieuw gevuld zodra het te bewerken recept verandert of (in Deel 8) binnenkomt vanaf de server.
- `form(this.model, (path) => { ... })` maakt de formulierboom. De tweede parameter is het **schema**: `path.title` is een **pad** naar een veld, waarop je regels zet.
- **Validatieregels** zijn gewone functies: `required`, `minLength`, `min`, `max`. Elk krijgt het pad en een optioneel `message`.
- `applyEach(path.ingredients, (item) => ...)` past regels toe op **elk element** van een lijst, ook als je er later elementen bijzet.

Toelichting: de template

- `[formField]="recipeForm.title"` koppelt een invoerveld aan een veld van het formulier. Er zijn geen `ngModel`, `value` of `(input)` meer nodig: de directive regelt tweerichtingsverkeer. Het werkt met `text`, `number`, `select` en `textarea`.
- `recipeForm.title()` (met haakjes) geeft de **status** van het veld: `touched()`, `invalid()`, `errors()`, `value()` en meer. Deze zijn allemaal signals.
- `@for (ingredient of recipeForm.ingredients; track $index)` loopt over de velden van de lijst; `ingredient.name` is het veld "naam" van dat ingrediënt.
- De knop "Ingrediënt toevoegen" past het **model** aan met `update` en een nieuwe array. Het formulier volgt vanzelf.

Toelichting: opslaan

- `submit(this.recipeForm, async () => { ... })` markeert alle velden als bezocht (`touched`), zodat alle foutmeldingen verschijnen, en voert de functie **alleen uit als het formulier geldig is**.
- In de functie lees je het model (`this.model()`) en sla je het op via de store, waarna je naar de detailpagina navigeert. De signals `saving` en `saveError` houden bij of we bezig zijn en of er iets misging. Dat lijkt nu overbodig (de store werkt in het geheugen), maar zodra we in Deel 8 naar een echte server schrijven, kan opslaan wél mislukken.
- `novalidate` op `<form>` schakelt de eigen browservalidatie uit, zodat onze meldingen getoond worden.

Stap 4: De routes aanvullen

Bestand: src/app/app.routes.ts

```
import { Routes } from '@angular/router';

export const routes: Routes = [
  { path: '', pathMatch: 'full', redirectTo: 'recepten' },
  {
    path: 'recepten',
    title: 'Recepten',
    loadComponent: () => import('./recipes/recipe-list/recipe-list').then((m) => m.RecipeList),
  },
  {
    path: 'recepten/nieuw',
    title: 'Nieuw recept',
    loadComponent: () => import('./recipes/recipe-form/recipe-form').then((m) => m.RecipeForm),
  },
  {
```

```
    path: 'recepten/:id/bewerken',
    title: 'Recept bewerken',
    loadComponent: () => import('./recipes/recipe-form/recipe-form').then((m) => m.RecipeForm),
  },
  {
    path: 'recepten/:id',
    title: 'Recept',
    loadComponent: () => import('./recipes/recipe-detail/recipe-detail').then((m) => m.RecipeDetail
),
  },
  {
    path: '**',
    title: 'Niet gevonden',
    loadComponent: () => import('./shared/not-found/not-found').then((m) => m.NotFound),
  },
];
```

Toelichting

- `recepten/nieuw` staat **vóór** `recepten/:id`. Anders zou de router "nieuw" als id opvatten. Volgorde telt altijd.
- Beide formulierroutes gebruiken hetzelfde component. Op `recepten/:id/bewerken` krijgt het de `id` als input; op `recepten/nieuw` is `id` `undefined`.

Stap 5: Links naar het formulier

Voeg in de navigatie een link toe en in de detailpagina een "Bewerken"-knop.

Bestand: src/app/app.html

```
<div class="min-h-screen bg-stone-50">
  <header class="border-b border-stone-200 bg-white">
    <div class="mx-auto flex max-w-5xl flex-wrap items-center justify-between gap-3 px-6 py-4">
      <a routerLink="/recepten" class="text-xl font-bold text-orange-700">Weekmenu</a>

      <nav class="flex gap-1">
        <a
          routerLink="/recepten"
          routerLinkActive="bg-orange-100 text-orange-800"
          [routerLinkActiveOptions]="{ exact: true }"
          [class]="link"
        >
          Recepten
        </a>
        <a routerLink="/recepten/nieuw" routerLinkActive="bg-orange-100 text-orange-800"
[class]="link">
          Nieuw recept
        </a>
      </nav>
    </div>
  </header>

  <main class="mx-auto max-w-5xl p-6">
    <router-outlet />
  </main>
</div>
```

Vervang in `recipe-detail.html` het blok met de verwijderknop onderaan door twee knoppen. Dit is het volledige bestand:

Bestand: src/app/recipes/recipe-detail/recipe-detail.html

```
@if (recipe(); as r) {
  <div class="space-y-6">
```

```
    <a routerLink="/recepten" class="text-sm text-orange-700 hover:underline">Terug naar alle
recepten</a>

    <article class="rounded-xl border border-stone-200 bg-white p-6 shadow-sm">
      <div class="flex flex-wrap items-start justify-between gap-3">
        <div>
          <h1 class="text-2xl font-bold">{{ r.title }}</h1>
          <p class="text-stone-600">{{ r.description }}</p>
        </div>
        <span class="text-sm text-stone-500">{{ r.minutes }} minuten</span>
      </div>

      <div class="mt-6 flex items-center gap-3">
        <span class="text-sm font-medium text-stone-700">Aantal personen</span>
        <button type="button" (click)="change(-1)" class="h-8 w-8 rounded-lg border border-
stone-300 hover:bg-stone-100">-</button>
        <span class="w-6 text-center font-semibold">{{ servings() }}</span>
        <button type="button" (click)="change(1)" class="h-8 w-8 rounded-lg border border-
stone-300 hover:bg-stone-100">+</button>
      </div>

      <h2 class="mt-6 text-lg font-semibold">Ingrediënten</h2>
      <ul class="mt-2">
        @for (item of ingredients(); track $index) {
          <li class="flex justify-between border-b border-stone-100 py-1">
            <span>{{ item.name }}</span>
            <span class="text-stone-600">{{ item.text }}</span>
          </li>
        }
      </ul>

      <h2 class="mt-6 text-lg font-semibold">Bereiding</h2>
      <ol class="mt-2 list-decimal space-y-1 pl-5">
        @for (step of steps(); track $index) {
          <li>{{ step }}</li>
        }
      </ol>
    </article>

    <div class="flex items-center gap-4">
      <a
        [routerLink]="['/recepten', r.id, 'bewerken']"
        class="rounded-lg border border-stone-300 px-4 py-2 text-sm font-medium text-stone-700
hover:bg-stone-100"
      >
        Bewerken
      </a>
      <button
        type="button"
        (click)="remove(r.id)"
        class="text-sm text-stone-500 underline hover:text-red-700"
      >
        Recept verwijderen
      </button>
    </div>
  </div>
} @else {
  <p class="text-stone-600">
    Dit recept bestaat niet (meer).
    <a routerLink="/recepten" class="text-orange-700 hover:underline">Terug naar alle recepten</a>
  </p>
}
```

Test

1. Ga naar "Nieuw recept" en klik direct op Opslaan: alle velden met fouten tonen een melding.
2. Voeg een recept toe met meerdere ingrediënten, verwijder er één en sla op.
3. Bewerk een bestaand recept via de detailpagina.

> **Let op:** de data staat nu nog in het geheugen. Na een verversing van de pagina is je werk weg. In Deel 7 en 8 zetten we een echte database erachter.

Deel 6: RxJS, Reactive Forms en effecten

MODULE 17: Wat is RxJS en waarom kom je het overal tegen?

RxJS is een bibliotheek om met **stromen van gebeurtenissen over tijd** te werken. Een stroom heet een **Observable**. Angular gebruikt hem op veel plaatsen: HTTP-verzoeken, de router, `valueChanges` van formulieren.

Een Observable is een **lazy** belofte voor meerdere waarden: er gebeurt niets totdat iemand zich **abonneert** (`subscribe`).

Begrip	Betekenis
`Observable<T>`	een stroom waarden van het type `T`
`subscribe(...)`	luisteren naar de stroom; je ontvangt waarden, een fout of "klaar"
`pipe(...)`	operators aan elkaar koppelen om de stroom te bewerken
`map`	elke waarde omzetten
`filter`	waarden weggooien die niet voldoen
`debounceTime(ms)`	wachten tot er `ms` milliseconden niets meer binnenkomt
`distinctUntilChanged()`	dezelfde waarde twee keer na elkaar negeren
`switchMap`	bij elke nieuwe waarde een **nieuwe** stroom starten en de vorige annuleren
`catchError`	een fout opvangen en een vervangende stroom teruggeven
`tap`	iets doen zonder de waarde te veranderen (bijvoorbeeld loggen)

Signals of RxJS?

- **Signals** zijn voor **toestand**: een waarde die er altijd is en verandert (een teller, een lijst, een filter).
- **Observables** zijn voor **gebeurtenissen en tijd**: toetsaanslagen met vertraging, HTTP-verzoeken, websockets.
- Je schakelt ertussen met `toSignal(observable)` en `toObservable(signal)` uit `@angular/core/rxjs-interop`.

Tip: abonneer je zelden zelf met `subscribe`. Gebruik `toSignal`, de `async`-pipe in een template, of de `httpResource` uit Deel 7. Een vergeten `subscribe` is een veelvoorkomende bron van geheugenlekken.

> **Vergelijking met Vue/React:** Vue en React hebben geen ingebouwde Observables; je gebruikt `watch` of effecten. In Angular is RxJS een vast onderdeel van het ecosysteem. In een sollicitatie wordt er vrijwel altijd naar gevraagd.

MODULE 18: Een zoekveld met Reactive Forms en RxJS

Doel van deze module

Een zoekveld dat pas zoekt als de gebruiker even stopt met typen, gebouwd met een `FormControl`, `valueChanges` en `toSignal`.

Stap 1: recipe-list.ts

Bestand: src/app/recipes/recipe-list/recipe-list.ts

```ts
import { Component, computed, effect, inject, signal } from '@angular/core';
import { toSignal } from '@angular/core/rxjs-interop';
import { FormControl, ReactiveFormsModule } from '@angular/forms';
import { Router } from '@angular/router';
import { debounceTime, distinctUntilChanged, map } from 'rxjs';
import { Panel } from '../../shared/panel/panel';
import { CATEGORIES, Category } from '../recipe';
import { RecipeCard } from '../recipe-card/recipe-card';
import { RecipeStore } from '../recipe-store';

const STORAGE_KEY = 'weekmenu:category';

function loadCategory(): Category | 'alle' {
  try {
    const stored = localStorage.getItem(STORAGE_KEY);
    return CATEGORIES.find((category) => category === stored) ?? 'alle';
  } catch {
    return 'alle';
  }
}

@Component({
  selector: 'app-recipe-list',
  imports: [Panel, RecipeCard, ReactiveFormsModule],
  templateUrl: './recipe-list.html',
})
export class RecipeList {
  private readonly store = inject(RecipeStore);
  private readonly router = inject(Router);

  protected readonly categories = CATEGORIES;
  protected readonly recipes = this.store.recipes;
  protected readonly category = signal<Category | 'alle'>(loadCategory());

  // Reactive Forms: één los invoerveld als FormControl.
  protected readonly search = new FormControl('', { nonNullable: true });

  // RxJS: wacht 250 ms na de laatste toetsaanslag en zet het resultaat om naar een signal.
  private readonly query = toSignal(
    this.search.valueChanges.pipe(
      debounceTime(250),
      map((value) => value.trim().toLowerCase()),
      distinctUntilChanged(),
    ),
    { initialValue: '' },
  );

  protected readonly visible = computed(() => {
    const selected = this.category();
    const query = this.query();

    return this.recipes().filter(
      (recipe) =>
        (selected === 'alle' || recipe.category === selected) &&
        (query === '' || recipe.title.toLowerCase().includes(query)),
    );
```

```
  });

  protected readonly chip = 'rounded-full border border-stone-300 px-3 py-1 text-sm capitalize
text-stone-700 hover:bg-stone-100';
  protected readonly chipActive = 'rounded-full border border-orange-600 bg-orange-600 px-3 py-1
text-sm capitalize text-white';

  constructor() {
    // Effect: bewaar de gekozen categorie in de browser (een neveneffect).
    effect(() => {
      try {
        localStorage.setItem(STORAGE_KEY, this.category());
      } catch {
        // opslag niet beschikbaar: negeren
      }
    });
  }

  protected select(category: Category | 'alle'): void {
    this.category.set(category);
  }

  protected open(id: number): void {
    this.router.navigate(['/recepten', id]);
  }
}
```

Stap 2: recipe-list.html

Bestand: src/app/recipes/recipe-list/recipe-list.html

```
<div class="space-y-6">
  <app-panel title="Zoeken en filteren">
    <span panel-actions class="text-sm text-stone-500">
      {{ visible().length }} van {{ recipes().length }} recepten
    </span>

    <input
      type="search"
      [formControl]="search"
      placeholder="Zoek op titel"
      class="mb-3 w-full rounded-lg border border-stone-300 px-3 py-2 text-sm focus:border-
orange-500 focus:outline-none focus:ring-2 focus:ring-orange-200"
    />

    <div class="flex flex-wrap gap-2">
      <button type="button" (click)="select('alle')" [class]="category() === 'alle' ?
chipActive : chip">
        Alle
      </button>

      @for (item of categories; track item) {
        <button type="button" (click)="select(item)" [class]="category() === item ?
chipActive : chip">
          {{ item }}
        </button>
      }
    </div>
  </app-panel>

  <div class="grid gap-4 sm:grid-cols-2 lg:grid-cols-3">
    @for (recipe of visible(); track recipe.id) {
      <app-recipe-card [recipe]="recipe" (open)="open($event)" />
    } @empty {
      <p class="col-span-full rounded-xl border border-dashed border-stone-300 p-8 text-center
text-stone-500">
        Geen recepten gevonden.
      </p>
    }
```

```
  </div>
</div>
```

Toelichting: Reactive Forms

- Een `FormControl` is **één invoerveld als object**. `{ nonNullable: true }` zorgt dat de waarde altijd een `string` is (niet `string | null`).
- Je koppelt hem aan het veld met `[formControl]="search"`. Daarvoor importeer je `ReactiveFormsModule`.
- `valueChanges` is een **Observable** die bij elke wijziging de nieuwe waarde uitzendt.
- Voor grotere formulieren combineer je meerdere controls in een `FormGroup` of bouw je ze met `FormBuilder` (zie Deel 7, het loginformulier).

Toelichting: de pipeline

- `debounceTime(250)`: alleen doorgeven als er 250 ms niets meer getypt is.
- `map(...)`: spaties verwijderen en kleine letters.
- `distinctUntilChanged()`: niets doen als de zoekterm niet echt veranderd is.
- `toSignal(..., { initialValue: '' })` zet de Observable om naar een **signal** die je in `computed` kunt gebruiken. Hij abonneert en **ontabonneert zelf** wanneer het component verdwijnt: geen geheugenlekken.

Toelichting: effect()

- `effect(() => { ... })` voert code uit **elke keer dat een signal die erin gelezen wordt verandert**. Hier: `this.category()` wordt gelezen, dus bij elke andere categorie wordt hij opgeslagen.
- Je maakt een `effect` in de **constructor** of een veldinitialisatie (een zogenaamde injection context).
- **Gebruik `effect` alleen voor neveneffecten** (opslaan, loggen, DOM-API's). Een waarde afleiden doe je met `computed` of `linkedSignal`. Als je in een effect een signal zet die je ergens anders nodig hebt, is dat bijna altijd een teken dat een `computed` beter is.

Effect, computed of linkedSignal?

Ik wil…	Gebruik
een waarde die **volgt** uit andere signals	`computed`
een waarde die uit andere signals begint, maar die de gebruiker ook mag wijzigen	`linkedSignal`
iets **buiten** Angular doen als een signal verandert	`effect`

Test

Typ in het zoekveld: de lijst filtert pas als je even stopt. Kies een categorie, ververs de pagina: dezelfde categorie is nog gekozen.

Oefeningen

1. Voeg een tweede `FormControl` toe voor "maximale bereidingstijd" (een getal) en filter daar ook op.
2. Maak met `toSignal` en `valueChanges` een teller die toont hoe vaak de zoekterm echt veranderd is.

3. Probeer in plaats van `computed` een `effect` om `visible` te vullen. Wat gaat er mis (of voelt onhandig)? Dit laat zien waarom `computed` hier de juiste keuze is.

Deel 7: Supabase en authenticatie

Tot nu toe staan de recepten in het geheugen. In dit deel richt je een database in en laat je gebruikers inloggen. In Deel 8 koppel je de recepten eraan met `HttpClient`.

MODULE 19: Een Supabase-project met beveiligde tabellen

Stap 1: Project aanmaken

1. Ga naar **supabase.com** en log in (een gratis account volstaat).
2. Maak een nieuw project met de naam `weekmenu`, kies een sterk databasewachtwoord en een regio dicht bij je (bijvoorbeeld een Europese regio).
3. Wacht tot het project klaar is.

> **Let op:** gratis Supabase-projecten kunnen na een periode zonder gebruik gepauzeerd worden. Je hervat ze met één klik in het dashboard.

Stap 2: Tabellen, beveiliging en regels

Open in het dashboard de **SQL Editor**, plak onderstaande SQL en voer ze uit:

```
create table recipes (
  id bigint generated always as identity primary key,
  user_id uuid not null default auth.uid() references auth.users (id) on delete cascade,
  title text not null,
  description text not null default '',
  category text not null check (category in ('ontbijt', 'lunch', 'diner', 'dessert')),
  servings integer not null check (servings between 1 and 24),
  minutes integer not null check (minutes between 1 and 600),
  ingredients jsonb not null default '[]'::jsonb,
  steps text not null default '',
  created_at timestamptz not null default now()
);

create table meal_plan (
  id bigint generated always as identity primary key,
  user_id uuid not null default auth.uid() references auth.users (id) on delete cascade,
  day date not null,
  meal text not null check (meal in ('lunch', 'diner')),
  recipe_id bigint not null references recipes (id) on delete cascade,
  unique (user_id, day, meal)
);

alter table recipes enable row level security;
alter table meal_plan enable row level security;

create policy "eigen recepten" on recipes
  for all to authenticated
  using (user_id = (select auth.uid()))
  with check (user_id = (select auth.uid()));

create policy "eigen weekplanning" on meal_plan
  for all to authenticated
```

```
  using (user_id = (select auth.uid()))
  with check (user_id = (select auth.uid()));

grant select, insert, update, delete on recipes, meal_plan to authenticated;
```

Toelichting

- `user_id uuid ... default auth.uid()`: bij elke nieuwe rij vult de database **zelf** het id van de ingelogde gebruiker in. Je app hoeft dat nooit mee te sturen (en kan het dus ook niet vervalsen).
- `ingredients jsonb`: de lijst ingrediënten bewaren we als JSON in één kolom. Dat past bij ons model, waarin ingrediënten alleen bij één recept horen.
- `unique (user_id, day, meal)`: per dag en maaltijd één recept per gebruiker.
- `on delete cascade`: verwijder je een recept, dan verdwijnen ook de planning-regels ervan.
- **Row Level Security (RLS)** laat de **database** bepalen welke rijen je mag zien. Zonder policy kan niemand iets. De policies hierboven zeggen: een ingelogde gebruiker (`authenticated`) mag alleen rijen lezen en schrijven waarvan `user_id` zijn eigen id is.
- De laatste regel (`grant`) geeft de rol `authenticated` toegang tot de tabellen. In veel projecten gebeurt dat standaard, maar het kan geen kwaad om het zelf te doen. Krijg je later "permission denied" fouten, dan ontbrak dit.
- **Waarom is dit zo belangrijk?** In Deel 8 praat je app **rechtstreeks vanuit de browser** met de database. Alles wat in de browser staat kan iemand bekijken. De beveiliging zit dus niet in je Angular-code, maar in de database zelf. Dat is precies wat RLS doet.

Stap 3: E-mailbevestiging

Standaard moet iemand na het registreren een bevestigingsmail aanklikken. Tijdens het oefenen is dat lastig. In het dashboard vind je onder **Authentication** (bij de instellingen van de e-mailprovider) de optie om "Confirm email" uit te zetten. Laat je hem aan, dan werkt de app ook: de gebruiker klikt eerst op de link in de mail. De loginpagina die je zo bouwt, toont daarvoor een melding.

Zoek onder **Authentication** ook de instelling voor de **Site URL** en zet die op `http://localhost:4200`. Na het deployen (Deel 12) voeg je het echte adres toe.

Stap 4: De sleutels

Je hebt twee waarden nodig:

1. de **Project URL** (ziet eruit als `https://abcdefgh.supabase.co`);
2. de **publishable key** (begint met `sb_publishable_...`).

Je vindt ze in het dashboard onder **Project Settings**, bij **API Keys**. Oudere projecten tonen een `anon`-sleutel; die werkt op dezelfde manier.

> **Belangrijk: welke sleutel mag in de browser?** De **publishable** (of `anon`) sleutel is **bedoeld** om in de browser te staan: ze geeft alleen toegang binnen de regels van je RLS-policies. De **secret** (of `service_role`) sleutel omzeilt RLS volledig en mag **nooit** in een Angular-app, nooit in Git en nooit in de browser. Gebruik in deze cursus alleen de publishable sleutel.

MODULE 20: Environments en de AuthStore

Stap 1: Het pakket installeren

```
npm install @supabase/supabase-js
```

`@supabase/supabase-js` regelt het inloggen, de sessie en het vernieuwen van tokens voor je. We gebruiken het in deze cursus voor **authenticatie**; de gegevens zelf halen we in Deel 8 op met Angular's eigen `HttpClient`.

Stap 2: Environment-bestanden genereren

```
npx ng generate environments
```

Toelichting

- De CLI maakt `src/environments/environment.ts` (voor de productiebuild) en `environment.development.ts` (voor `npm start`) en past `angular.json` aan, zodat bij een ontwikkelbuild het ene bestand door het andere vervangen wordt (`fileReplacements`).
- Je importeert altijd `environment.ts`; de CLI zorgt voor de rest.

Stap 3: Je gegevens invullen

Zet in **beide** bestanden dezelfde inhoud, met jouw waarden:

Bestand: src/environments/environment.ts

```ts
export const environment = {
  supabaseUrl: 'https://JOUW-PROJECT.supabase.co',
  supabaseKey: 'sb_publishable_JOUW_SLEUTEL',
};
```

Bestand: src/environments/environment.development.ts

```ts
export const environment = {
  supabaseUrl: 'https://JOUW-PROJECT.supabase.co',
  supabaseKey: 'sb_publishable_JOUW_SLEUTEL',
};
```

Toelichting

- Omdat dit een publieke sleutel is, mag ze in je code staan en in Git. Niemand kan ermee meer dan de RLS-policies toestaan.
- Voor een echte organisatie gebruik je meestal een apart Supabase-project voor ontwikkeling en productie, en verschillen de twee bestanden.

Stap 4: Een gedeeld pad naar de API

Bestand: src/app/core/api.ts

```ts
import { environment } from '../../environments/environment';

// Het REST-adres van de database (PostgREST).
export const REST_URL = `${environment.supabaseUrl}/rest/v1`;
```

Stap 5: De AuthStore

```
npx ng generate service core/auth-store --skip-tests
```

Bestand: src/app/core/auth-store.ts

```ts
import { computed, Service, signal } from '@angular/core';
import { createClient, Session } from '@supabase/supabase-js';
import { environment } from '../../environments/environment';

@Service()
export class AuthStore {
  private readonly client = createClient(environment.supabaseUrl, environment.supabaseKey);
  private readonly sessionState = signal<Session | null>(null);

  // Een belofte die klaar is zodra de opgeslagen sessie gelezen is.
  private readonly initialized: Promise<void>;

  readonly user = computed(() => this.sessionState()?.user ?? null);
  readonly accessToken = computed(() => this.sessionState()?.access_token ?? null);
  readonly isLoggedIn = computed(() => this.sessionState() !== null);

  constructor() {
    this.initialized = this.client.auth.getSession().then(({ data }) => {
      this.sessionState.set(data.session);
    });

    // Wordt aangeroepen bij inloggen, uitloggen en het vernieuwen van de sessie.
    this.client.auth.onAuthStateChange((_event, session) => {
      this.sessionState.set(session);
    });
  }

  whenReady(): Promise<void> {
    return this.initialized;
  }

  async signIn(email: string, password: string): Promise<string | null> {
    const { error } = await this.client.auth.signInWithPassword({ email, password });
    return error?.message ?? null;
  }

  async signUp(email: string, password: string): Promise<{ error: string | null;
needsConfirmation: boolean }> {
    const { data, error } = await this.client.auth.signUp({ email, password });
    return { error: error?.message ?? null, needsConfirmation: !error && data.session === null };
  }

  async signOut(): Promise<void> {
    await this.client.auth.signOut();
  }
}
```

Toelichting

- De store verpakt de Supabase-client en geeft de sessie vrij als **signals**. De rest van de app weet niets van Supabase: ze gebruikt `user()`, `isLoggedIn()` en `accessToken()`.
- Bij het opstarten leest `getSession()` de **bewaarde sessie** (Supabase bewaart die in `localStorage`). Dat is asynchroon; daarom bewaren we de belofte (`initialized`) en laten guards erop wachten via `whenReady()`.
- `onAuthStateChange` houdt de signal actueel: logt iemand in of uit, of wordt het token vernieuwd, dan werkt de hele app mee.
- `signIn` en `signUp` geven een **foutmelding als tekst** terug (of `null` bij succes), zodat het formulier ze kan tonen.
- Bij `signUp` is `session` leeg als e-mailbevestiging aan staat: dan moet de gebruiker eerst op de link in de mail klikken.

MODULE 21: De loginpagina met Reactive Forms

Doel van deze module

Een formulier met `FormBuilder`, `Validators` en foutmeldingen, gebouwd met de **klassieke Reactive Forms**. Dit is de variant die je in de meeste bestaande Angular-projecten tegenkomt.

Stap 1: Genereren

```
npx ng generate component auth/login --skip-tests --style=none
```

Stap 2: login.ts

Bestand: src/app/auth/login/login.ts

```
import { Component, inject, input, signal } from '@angular/core';
import { NonNullableFormBuilder, ReactiveFormsModule, Validators } from '@angular/forms';
import { Router } from '@angular/router';
import { AuthStore } from '../../core/auth-store';

@Component({
  selector: 'app-login',
  imports: [ReactiveFormsModule],
  templateUrl: './login.html',
})
export class Login {
  private readonly auth = inject(AuthStore);
  private readonly router = inject(Router);
  private readonly fb = inject(NonNullableFormBuilder);

  // Query-parameter ?terug=/pad: ook die komt dankzij withComponentInputBinding als input binnen.
  readonly terug = input<string>();

  protected readonly mode = signal<'inloggen' | 'registreren'>('inloggen');
  protected readonly busy = signal(false);
  protected readonly error = signal<string | null>(null);
  protected readonly info = signal<string | null>(null);

  protected readonly form = this.fb.group({
    email: ['', [Validators.required, Validators.email]],
    password: ['', [Validators.required, Validators.minLength(6)]],
```

```
  });

  protected toggle(): void {
    this.mode.update((mode) => (mode === 'inloggen' ? 'registreren' : 'inloggen'));
    this.error.set(null);
    this.info.set(null);
  }

  protected async submit(): Promise<void> {
    if (this.form.invalid) {
      this.form.markAllAsTouched();
      return;
    }

    this.busy.set(true);
    this.error.set(null);
    this.info.set(null);

    const { email, password } = this.form.getRawValue();

    try {
      if (this.mode() === 'inloggen') {
        const error = await this.auth.signIn(email, password);
        if (error) {
          this.error.set(error);
          return;
        }
        await this.router.navigateByUrl(this.target());
      } else {
        const result = await this.auth.signUp(email, password);
        if (result.error) {
          this.error.set(result.error);
          return;
        }
        if (result.needsConfirmation) {
          this.info.set('Account aangemaakt. Klik op de link in de e-mail om te bevestigen en log
daarna in.');
          return;
        }
        await this.router.navigateByUrl('/recepten');
      }
    } finally {
      this.busy.set(false);
    }
  }

  // Alleen interne paden toestaan, anders kan iemand je na het inloggen naar een vreemde site
sturen.
  private target(): string {
    const target = this.terug();
    return target && target.startsWith('/') && !target.startsWith('//') ? target : '/recepten';
  }
}
```

Stap 3: login.html

Bestand: src/app/auth/login/login.html

```
<div class="mx-auto max-w-sm space-y-4">
  <h1 class="text-2xl font-bold">{{ mode() === 'inloggen' ? 'Inloggen' : 'Account maken' }}</h1>

  <form
    [formGroup]="form"
    (ngSubmit)="submit()"
    novalidate
    class="space-y-4 rounded-xl border border-stone-200 bg-white p-6 shadow-sm"
  >
    @if (error(); as message) {
      <p role="alert" class="rounded-lg bg-red-50 px-3 py-2 text-sm text-red-800">{{ message }}</p>
    }
    @if (info(); as message) {
```

```
      <p class="rounded-lg bg-emerald-50 px-3 py-2 text-sm text-emerald-800">{{ message }}</p>
    }

    <div>
      <label for="email" class="mb-1 block text-sm font-medium text-stone-700">E-mailadres</label>
      <input
        id="email"
        type="email"
        formControlName="email"
        autocomplete="email"
        class="w-full rounded-lg border border-stone-300 px-3 py-2 text-sm focus:border-orange-500
focus:outline-none focus:ring-2 focus:ring-orange-200"
      />
      @if (form.controls.email.touched && form.controls.email.invalid) {
        <p class="mt-1 text-sm text-red-700">Vul een geldig e-mailadres in.</p>
      }
    </div>

    <div>
      <label for="password" class="mb-1 block text-sm font-medium text-stone-700">Wachtwoord</
label>
      <input
        id="password"
        type="password"
        formControlName="password"
        [autocomplete]="mode() === 'inloggen' ? 'current-password' : 'new-password'"
        class="w-full rounded-lg border border-stone-300 px-3 py-2 text-sm focus:border-orange-500
focus:outline-none focus:ring-2 focus:ring-orange-200"
      />
      @if (form.controls.password.touched && form.controls.password.invalid) {
        <p class="mt-1 text-sm text-red-700">Minstens 6 tekens.</p>
      }
    </div>

    <button
      type="submit"
      [disabled]="busy()"
      class="w-full rounded-lg bg-orange-600 px-4 py-2 font-medium text-white hover:bg-orange-700
disabled:opacity-60"
    >
      {{ busy() ? 'Bezig...' : mode() === 'inloggen' ? 'Inloggen' : 'Registreren' }}
    </button>
  </form>

  <button type="button" (click)="toggle()" class="text-sm text-orange-700 hover:underline">
    {{ mode() === 'inloggen' ? 'Nog geen account? Registreer je.' : 'Al een account? Log in.' }}
  </button>
</div>
```

Toelichting: Reactive Forms

- `NonNullableFormBuilder` bouwt een **FormGroup** met een kortere schrijfwijze. Elk veld is een `FormControl` met een beginwaarde en een lijst **validators**. `Validators.required`, `.email` en `.minLength(6)` zijn ingebouwd.
- De formulierstructuur (`form`) is een **object in de klasse**, niet iets in de template. Daarom heet het "reactive": de klasse is de bron van waarheid.
- In de template koppel je: `[formGroup]="form"` op het `<form>`, `formControlName="email"` op elk veld, en `(ngSubmit)` voor het versturen.
- `form.controls.email.touched` en `.invalid` bepalen wanneer we een foutmelding tonen: pas nadat het veld bezocht is.
- `markAllAsTouched()` toont bij een mislukte poging alle fouten tegelijk.
- `getRawValue()` geeft een object `{ email, password }` met de juiste types.

Toelichting: de rest

- `terug` is een `input`: de router koppelt query-parameters (`?terug=/weekplanner`) net als padparameters aan inputs. We gebruiken hem zo meteen voor "ga terug naar waar je was".
- `target()` laat alleen interne paden toe. Zonder deze controle kon een aanvaller een link maken die je na het inloggen naar een andere site stuurt (een *open redirect*).
- `try/finally` zorgt dat `busy` altijd weer uitgaat, ook bij een `return` of fout.

> **Vergelijking:** Signal Forms (Deel 5) en Reactive Forms (hier) lossen hetzelfde op. Bij Signal Forms is de **data** een signal en de regels staan in een schema; bij Reactive Forms is het **formulier** een object van `FormControl`s. Beide zijn nuttig om te kennen: nieuwe code kiest bij voorkeur Signal Forms, bestaande code gebruikt bijna altijd Reactive Forms.

MODULE 22: Route guards en de header

Doel van deze module

Pagina's beveiligen: wie niet ingelogd is, wordt naar de loginpagina gestuurd.

Stap 1: De guards

```
npx ng generate guard core/auth --functional --skip-tests
```

(Kies als de CLI vraagt welk type: `CanActivate`.) Vervang de inhoud van `src/app/core/auth-guard.ts`:

Bestand: src/app/core/auth-guard.ts

```ts
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { AuthStore } from './auth-store';

// Alleen voor ingelogde gebruikers.
export const authGuard: CanActivateFn = async (_route, state) => {
  const auth = inject(AuthStore);
  const router = inject(Router);

  await auth.whenReady();

  return auth.isLoggedIn()
    ? true
    : router.createUrlTree(['/inloggen'], { queryParams: { terug: state.url } });
};

// Alleen voor bezoekers die nog niet ingelogd zijn (de loginpagina).
export const guestGuard: CanActivateFn = async () => {
  const auth = inject(AuthStore);
  const router = inject(Router);

  await auth.whenReady();

  return auth.isLoggedIn() ? router.createUrlTree(['/recepten']) : true;
};
```

Toelichting

- Een guard is een **functie** (`CanActivateFn`) die bepaalt of een route geopend mag worden. Ze mag `true`, `false`, een `UrlTree` (een omleiding) of een belofte/Observable daarvan teruggeven.
- `inject()` werkt in een guard, omdat Angular de functie in een injectiecontext uitvoert.
- `await auth.whenReady()`: bij een pagina-verversing is de sessie nog niet gelezen. Zonder te wachten zou je altijd naar de loginpagina worden gestuurd.
- `router.createUrlTree([...], { queryParams })` bouwt de omleiding, en zet de oorspronkelijke URL in `?terug=...`. De loginpagina leest die.

Stap 2: De routes

Bestand: src/app/app.routes.ts

```
import { Routes } from '@angular/router';
import { authGuard, guestGuard } from './core/auth-guard';

export const routes: Routes = [
  {
    path: 'inloggen',
    title: 'Inloggen',
    canActivate: [guestGuard],
    loadComponent: () => import('./auth/login/login').then((m) => m.Login),
  },
  {
    path: '',
    canActivate: [authGuard],
    children: [
      { path: '', pathMatch: 'full', redirectTo: 'recepten' },
      {
        path: 'recepten',
        title: 'Recepten',
        loadComponent: () => import('./recipes/recipe-list/recipe-list').then((m) => m.RecipeList),
      },
      {
        path: 'recepten/nieuw',
        title: 'Nieuw recept',
        loadComponent: () => import('./recipes/recipe-form/recipe-form').then((m) => m.RecipeForm),
      },
      {
        path: 'recepten/:id/bewerken',
        title: 'Recept bewerken',
        loadComponent: () => import('./recipes/recipe-form/recipe-form').then((m) => m.RecipeForm),
      },
      {
        path: 'recepten/:id',
        title: 'Recept',
        loadComponent: () => import('./recipes/recipe-detail/recipe-detail').then((m) => m.RecipeDe
tail),
      },
    ],
  },
  {
    path: '**',
    title: 'Niet gevonden',
    loadComponent: () => import('./shared/not-found/not-found').then((m) => m.NotFound),
  },
];
```

Toelichting

- Een route **zonder component maar met `children`** is een groepering. De `authGuard` op de ouder geldt voor **alle** kinderen. Zo beveilig je een heel gebied in één regel.

- Het volgorde-principe blijft gelden: `inloggen` eerst, dan de beveiligde groep, en de wildcard `**` als allerlaatste.

Stap 3: De header met uitloggen

Bestand: src/app/app.ts

```
import { Component, inject } from '@angular/core';
import { Router, RouterLink, RouterLinkActive, RouterOutlet } from '@angular/router';
import { AuthStore } from './core/auth-store';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet, RouterLink, RouterLinkActive],
  templateUrl: './app.html',
})
export class App {
  protected readonly auth = inject(AuthStore);
  private readonly router = inject(Router);

  protected readonly link = 'rounded-lg px-3 py-2 text-sm font-medium text-stone-600 hover:bg-
stone-100';

  protected async logout(): Promise<void> {
    await this.auth.signOut();
    await this.router.navigateByUrl('/inloggen');
  }
}
```

Bestand: src/app/app.html

```
<div class="min-h-screen bg-stone-50">
  <header class="border-b border-stone-200 bg-white">
    <div class="mx-auto flex max-w-5xl flex-wrap items-center justify-between gap-3 px-6 py-4">
      <a routerLink="/recepten" class="text-xl font-bold text-orange-700">Weekmenu</a>

      @if (auth.isLoggedIn()) {
        <nav class="flex flex-wrap items-center gap-1">
          <a
            routerLink="/recepten"
            routerLinkActive="bg-orange-100 text-orange-800"
            [routerLinkActiveOptions]="{ exact: true }"
            [class]="link"
          >
            Recepten
          </a>
          <a routerLink="/recepten/nieuw" routerLinkActive="bg-orange-100 text-orange-800"
[class]="link">
            Nieuw recept
          </a>

          <span class="ml-3 hidden text-sm text-stone-500 sm:inline">{{ auth.user()?.email }}</
span>
          <button type="button" (click)="logout()" [class]="link">Uitloggen</button>
        </nav>
      }
    </div>
  </header>

  <main class="mx-auto max-w-5xl p-6">
    <router-outlet />
  </main>
</div>
```

Toelichting

- De navigatie verschijnt alleen voor ingelogde gebruikers (`@if (auth.isLoggedIn())`), en toont het e-mailadres met `auth.user()?.email` (de `?.` voorkomt een fout als er geen gebruiker is).
- `protected readonly auth` is `protected` en niet `private`, omdat de **template** het gebruikt.

Test

1. Open `http://localhost:4200`: je wordt naar `/inloggen?terug=%2Frecepten` gestuurd.
2. Registreer een account (en bevestig eventueel je e-mail), log in: je komt op de recepten terecht.
3. Ververs de pagina: je blijft ingelogd.
4. Klik op Uitloggen en probeer `/recepten` te openen: je wordt weer naar het inloggen gestuurd.
5. Kijk in het dashboard onder Authentication naar de lijst gebruikers.

Deel 8: HTTP, httpResource en interceptors

MODULE 23: HttpClient en interceptors

Hoe praat Angular met een server?

Angular heeft een eigen HTTP-client: `HttpClient`. Je activeert hem met `provideHttpClient(...)` in `app.config.ts`. In Angular 22 gebruikt hij volgens de release-aankondiging standaard de moderne **Fetch API** onder de motorkap; voor jou verandert er niets aan hoe je hem gebruikt.

`HttpClient` geeft **Observables** terug: `this.http.get<Recipe[]>(url)`. Er gebeurt niets totdat iemand zich abonneert. Voor het **lezen** van gegevens gebruik je in Angular 22 liever `httpResource` (Module 25); voor **schrijven** (POST, PATCH, DELETE) gebruik je `HttpClient` direct.

Wat is een interceptor?

Een **interceptor** is een functie die **elk HTTP-verzoek** onderschept, voordat het de server bereikt, en elk antwoord, voordat de rest van je code het ziet. Typische toepassingen:

- headers toevoegen (zoals een token),
- fouten centraal afhandelen,
- laad-indicatoren,
- logging.

Wij hebben er twee nodig:

1. een interceptor die bij elk verzoek naar Supabase de **API-sleutel** en het **token van de ingelogde gebruiker** meestuurt;
2. een interceptor die een **401-fout** (niet meer ingelogd) afvangt.

MODULE 24: De twee interceptors

Stap 1: Genereren

```
npx ng generate interceptor core/supabase --skip-tests
npx ng generate interceptor core/error --skip-tests
```

Stap 2: supabase-interceptor.ts

Bestand: src/app/core/supabase-interceptor.ts

```
import { HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { environment } from '../../environments/environment';
import { AuthStore } from './auth-store';
```

```ts
// Voegt de sleutel en het token van de gebruiker toe aan elk verzoek naar Supabase.
export const supabaseInterceptor: HttpInterceptorFn = (req, next) => {
  if (!req.url.startsWith(environment.supabaseUrl)) {
    return next(req);
  }

  const token = inject(AuthStore).accessToken();

  let headers = req.headers.set('apikey', environment.supabaseKey);
  if (token) {
    headers = headers.set('Authorization', `Bearer ${token}`);
  }

  return next(req.clone({ headers }));
};
```

Toelichting

- Een interceptor is een gewone functie `(req, next) => Observable`. `next(req)` geeft het verzoek door aan de volgende schakel.
- **Verzoeken zijn onveranderlijk**: je past ze niet aan, maar maakt met `req.clone({ ... })` een kopie met aanpassingen.
- We voegen de headers **alleen** toe aan verzoeken naar onze eigen Supabase-URL. Stuur nooit een token mee naar andere sites.
- `apikey` identificeert je project; `Authorization: Bearer <token>` bewijst wie ingelogd is. Op basis van dat token weet de database wie `auth.uid()` is en past ze de RLS-policies toe.
- `inject(AuthStore)` werkt hier, omdat de interceptor in een injectiecontext draait. Roep `inject` wel **bovenaan** de functie aan, niet in een callback.

Stap 3: error-interceptor.ts

Bestand: src/app/core/error-interceptor.ts

```ts
import { HttpErrorResponse, HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { Router } from '@angular/router';
import { catchError, throwError } from 'rxjs';

// Bij een 401 (sessie verlopen) sturen we de gebruiker naar de loginpagina.
export const errorInterceptor: HttpInterceptorFn = (req, next) => {
  const router = inject(Router);

  return next(req).pipe(
    catchError((error: unknown) => {
      if (error instanceof HttpErrorResponse && error.status === 401) {
        router.navigateByUrl('/inloggen');
      }
      return throwError(() => error);
    }),
  );
};
```

Toelichting

- `catchError` uit RxJS vangt een fout in de stroom op. Wij sturen de fout daarna **opnieuw door** met `throwError`, zodat de code die het verzoek deed ook weet dat het mislukt is.
- `error instanceof HttpErrorResponse` controleert of het een HTTP-fout is en geeft toegang tot `error.status`.
- De volgorde van interceptors telt: ze worden in de volgorde van de lijst uitgevoerd.

Stap 4: Activeren in app.config.ts

Bestand: src/app/app.config.ts

```
import { provideHttpClient, withInterceptors } from '@angular/common/http';
import { ApplicationConfig, provideBrowserGlobalErrorListeners } from '@angular/core';
import { provideRouter, withComponentInputBinding } from '@angular/router';
import { routes } from './app.routes';
import { errorInterceptor } from './core/error-interceptor';
import { supabaseInterceptor } from './core/supabase-interceptor';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideRouter(routes, withComponentInputBinding()),
    provideHttpClient(withInterceptors([supabaseInterceptor, errorInterceptor])),
  ],
};
```

MODULE 25: De RecipeStore met httpResource

Doel van deze module

De store haalt recepten op van de database en schrijft wijzigingen weg. De componenten hoeven bijna niet te veranderen: de **namen** van de methoden blijven gelijk.

httpResource in het kort

`httpResource(() => request)` is een **signal-gebaseerde** manier om data op te halen:

- je geeft een functie die het **verzoek** beschrijft (de URL en eventuele parameters);
- gebruikt die functie signals, dan wordt het verzoek **automatisch opnieuw gedaan** als ze veranderen;
- geeft de functie `undefined` terug, dan wordt er **niets** opgehaald (handig als je nog niet ingelogd bent);
- je krijgt signals terug: `value()`, `isLoading()`, `error()`, `hasValue()`, plus `reload()`, `set()` en `update()`.

Dit vervangt de oude combinatie van `subscribe`, een loading-vlag en een error-vlag.

recipe-store.ts

Bestand: src/app/recipes/recipe-store.ts

```
import { HttpClient, httpResource } from '@angular/common/http';
import { computed, inject, Service } from '@angular/core';
import { map, Observable, tap } from 'rxjs';
import { REST_URL } from '../core/api';
import { AuthStore } from '../core/auth-store';
import { NewRecipe, Recipe } from './recipe';

// Vraag Supabase om de aangemaakte of gewijzigde rijen terug te sturen.
const RETURN_ROWS = { headers: { Prefer: 'return=representation' } };

function firstRow(rows: Recipe[]): Recipe {
  const row = rows[0];
  if (!row) throw new Error('De wijziging is niet opgeslagen.');
  return row;
}
```

```ts
@Service()
export class RecipeStore {
  private readonly http = inject(HttpClient);
  private readonly auth = inject(AuthStore);

  // Alleen ophalen als je ingelogd bent; opnieuw ophalen na in- of uitloggen gebeurt vanzelf.
  private readonly list = httpResource<Recipe[]>(() =>
    this.auth.isLoggedIn() ? { url: `${REST_URL}/recipes`, params: { select: '*' } } : undefined,
  );

  readonly recipes = computed(() => {
    const rows = this.list.hasValue() ? this.list.value() : [];
    return [...rows].sort((a, b) => a.title.localeCompare(b.title, 'nl'));
  });

  readonly loading = this.list.isLoading;
  readonly error = computed(() => (this.list.error() ? 'De recepten konden niet geladen worden.' :
null));

  byId(id: number): Recipe | undefined {
    return this.recipes().find((recipe) => recipe.id === id);
  }

  add(recipe: NewRecipe): Observable<Recipe> {
    return this.http.post<Recipe[]>(`${REST_URL}/recipes`, recipe, RETURN_ROWS).pipe(
      map(firstRow),
      tap((created) => this.list.update((current) => [...(current ?? []), created])),
    );
  }

  addMany(recipes: NewRecipe[]): Observable<Recipe[]> {
    return this.http.post<Recipe[]>(`${REST_URL}/recipes`, recipes, RETURN_ROWS).pipe(
      tap((created) => this.list.update((current) => [...(current ?? []), ...created])),
    );
  }

  update(id: number, changes: NewRecipe): Observable<Recipe> {
    return this.http.patch<Recipe[]>(`${REST_URL}/recipes?id=eq.${id}`, changes, RETURN_ROWS).pipe(
      map(firstRow),
      tap((updated) =>
        this.list.update((current) => (current ?? []).map((item) => (item.id === id ? updated : ite
m))),
      ),
    );
  }

  remove(id: number): Observable<void> {
    return this.http.delete<void>(`${REST_URL}/recipes?id=eq.${id}`).pipe(
      tap(() => this.list.update((current) => (current ?? []).filter((item) => item.id !== id))),
    );
  }
}
```

Toelichting: lezen

- Het verzoek (`GET /rest/v1/recipes?select=*`) wordt gedaan zodra `isLoggedIn()` waar is. De interceptor voegt de headers toe, en dankzij RLS komen alleen **jouw** recepten terug.
- `hasValue()` controleert of er data is (anders zou `value()` in een foutstatus een uitzondering kunnen geven). Zonder data tonen we een lege lijst.
- `recipes` sorteert op titel. Dat doen we in een `computed`, niet in de database, zodat lokale wijzigingen ook meteen goed gesorteerd staan.
- `loading` en `error` zijn signals voor de UI.

Toelichting: schrijven

- Supabase gebruikt **PostgREST**: de URL beschrijft de bewerking. `?id=eq.5` betekent "waar id gelijk is aan 5". `POST` maakt aan, `PATCH` wijzigt, `DELETE` verwijdert.
- De header `Prefer: return=representation` zorgt dat Supabase de aangemaakte rij **terugstuurt** (anders krijg je niets terug).
- Na een geslaagde wijziging werken we de lijst **lokaal** bij met `update`. De gebruiker ziet het resultaat meteen, zonder de hele lijst opnieuw op te halen.
- De methoden geven een `Observable` terug. **Er gebeurt niets** totdat iemand abonneert of `firstValueFrom` aanroept (zie hieronder). Dat is een klassieke valkuil: een vergeten abonnement betekent een verzoek dat nooit wordt verstuurd.

MODULE 26: De componenten aanpassen

Omdat de store nu Observables geeft, passen we de plekken aan die hem gebruiken. De hulpfunctie `firstValueFrom` uit RxJS wacht op de eerste waarde van een Observable en geeft hem terug als **belofte**, zodat je `await` kunt gebruiken.

recipe-form.ts

Bestand: src/app/recipes/recipe-form/recipe-form.ts

```
import { Component, computed, inject, input, linkedSignal, signal } from '@angular/core';
import { applyEach, form, FormField, max, min, minLength, required, submit } from '@angular/forms/
signals';
import { Router, RouterLink } from '@angular/router';
import { firstValueFrom } from 'rxjs';
import { FieldError, FormErrors } from '../../shared/form-errors/form-errors';
import { CATEGORIES, Category, Ingredient, NewRecipe, Recipe, UNITS } from '../recipe';
import { RecipeStore } from '../recipe-store';

interface RecipeModel {
  title: string;
  description: string;
  category: Category;
  servings: number;
  minutes: number;
  steps: string;
  ingredients: Ingredient[];
}

function emptyIngredient(): Ingredient {
  return { name: '', amount: 1, unit: 'g' };
}

function emptyModel(): RecipeModel {
  return {
    title: '',
    description: '',
    category: 'diner',
    servings: 4,
    minutes: 30,
    steps: '',
    ingredients: [emptyIngredient()],
  };
}

function toModel(recipe: Recipe): RecipeModel {
  return {
    title: recipe.title,
```

```
    description: recipe.description,
    category: recipe.category,
    servings: recipe.servings,
    minutes: recipe.minutes,
    steps: recipe.steps,
    ingredients: recipe.ingredients.map((ingredient) => ({ ...ingredient })),
  };
}

@Component({
  selector: 'app-recipe-form',
  imports: [FormField, FormErrors, RouterLink],
  templateUrl: './recipe-form.html',
})
export class RecipeForm {
  private readonly store = inject(RecipeStore);
  private readonly router = inject(Router);

  // Alleen aanwezig op de route /recepten/:id/bewerken
  readonly id = input<string>();

  protected readonly categories = CATEGORIES;
  protected readonly units = UNITS;

  protected readonly existing = computed(() => {
    const id = this.id();
    return id === undefined ? undefined : this.store.byId(Number(id));
  });

  // Het formuliermodel: leeg bij een nieuw recept, gevuld bij bewerken.
  protected readonly model = linkedSignal<RecipeModel>(() => {
    const recipe = this.existing();
    return recipe ? toModel(recipe) : emptyModel();
  });

  protected readonly recipeForm = form(this.model, (path) => {
    required(path.title, { message: 'Geef het recept een titel.' });
    minLength(path.title, 3, { message: 'De titel heeft minstens 3 tekens nodig.' });
    required(path.steps, { message: 'Beschrijf de bereiding.' });
    min(path.servings, 1, { message: 'Minstens 1 persoon.' });
    max(path.servings, 24, { message: 'Maximaal 24 personen.' });
    min(path.minutes, 1, { message: 'Minstens 1 minuut.' });
    max(path.minutes, 600, { message: 'Maximaal 600 minuten.' });
    minLength(path.ingredients, 1, { message: 'Voeg minstens één ingrediënt toe.' });

    applyEach(path.ingredients, (item) => {
      required(item.name, { message: 'Naam ontbreekt.' });
      min(item.amount, 0.01, { message: 'De hoeveelheid moet groter zijn dan 0.' });
    });
  });

  protected readonly saving = signal(false);
  protected readonly saveError = signal<string | null>(null);

  protected readonly input =
    'w-full rounded-lg border border-stone-300 bg-white px-3 py-2 text-sm focus:border-orange-500
focus:outline-none focus:ring-2 focus:ring-orange-200';
  protected readonly label = 'mb-1 block text-sm font-medium text-stone-700';

  // Toon fouten pas nadat het veld bezocht is.
  protected shown(state: { touched(): boolean; errors(): readonly FieldError[] }): readonly FieldEr
ror[] {
    return state.touched() ? state.errors() : [];
  }

  protected addIngredient(): void {
    this.model.update((current) => ({ ...current, ingredients: [...current.ingredients, emptyIngred
ient()] }));
  }

  protected removeIngredient(index: number): void {
    this.model.update((current) => ({
      ...current,
```

```
      ingredients: current.ingredients.filter((_, position) => position !== index),
    }));
  }

  protected async save(event: Event): Promise<void> {
    event.preventDefault();
    this.saveError.set(null);

    await submit(this.recipeForm, async () => {
      this.saving.set(true);

      try {
        const value: NewRecipe = { ...this.model() };
        const existing = this.existing();

        if (existing) {
          await firstValueFrom(this.store.update(existing.id, value));
          await this.router.navigate(['/recepten', existing.id]);
        } else {
          const created = await firstValueFrom(this.store.add(value));
          await this.router.navigate(['/recepten', created.id]);
        }
      } catch (error) {
        this.saveError.set(error instanceof Error ? error.message : 'Opslaan is mislukt.');
      } finally {
        this.saving.set(false);
      }
    });
  }
}
```

Toelichting: alleen het gedeelte in `save` is veranderd: `firstValueFrom(this.store.add(value))` start het verzoek, wacht op het antwoord en geeft het nieuwe recept terug. Gaat er iets mis, dan komt de fout in het `catch`-blok terecht en tonen we hem in de template (`saveError`).

recipe-detail.ts

Bestand: src/app/recipes/recipe-detail/recipe-detail.ts

```
import { Component, computed, inject, input, linkedSignal, signal } from '@angular/core';
import { Router, RouterLink } from '@angular/router';
import { firstValueFrom } from 'rxjs';
import { formatAmount, scaleAmount } from '../recipe';
import { RecipeStore } from '../recipe-store';

@Component({
  selector: 'app-recipe-detail',
  imports: [RouterLink],
  templateUrl: './recipe-detail.html',
})
export class RecipeDetail {
  private readonly store = inject(RecipeStore);
  private readonly router = inject(Router);

  // Komt uit de URL: /recepten/:id
  readonly id = input.required<string>();

  protected readonly loading = this.store.loading;
  protected readonly error = signal<string | null>(null);

  protected readonly recipe = computed(() => this.store.byId(Number(this.id())));

  // Begint bij het aantal personen van het recept en wordt opnieuw ingesteld als het recept
verandert.
  protected readonly servings = linkedSignal(() => this.recipe()?.servings ?? 1);

  protected readonly ingredients = computed(() => {
    const recipe = this.recipe();
    if (!recipe) return [];
```

```
    return recipe.ingredients.map((ingredient) => ({
      name: ingredient.name,
      text: formatAmount(scaleAmount(ingredient.amount, recipe.servings, this.servings()), ingredie
nt.unit),
    }));
  });

  protected readonly steps = computed(
    () => this.recipe()?.steps.split('\n').filter((line) => line.trim() !== '') ?? [],
  );

  protected change(delta: number): void {
    this.servings.update((value) => Math.min(24, Math.max(1, value + delta)));
  }

  protected async remove(id: number): Promise<void> {
    this.error.set(null);

    try {
      await firstValueFrom(this.store.remove(id));
      await this.router.navigateByUrl('/recepten');
    } catch {
      this.error.set('Het recept kon niet verwijderd worden.');
    }
  }
}
```

recipe-detail.html

Bestand: src/app/recipes/recipe-detail/recipe-detail.html

```
@if (recipe(); as r) {
  <div class="space-y-6">
    <a routerLink="/recepten" class="text-sm text-orange-700 hover:underline">Terug naar alle
recepten</a>

    <article class="rounded-xl border border-stone-200 bg-white p-6 shadow-sm">
      <div class="flex flex-wrap items-start justify-between gap-3">
        <div>
          <h1 class="text-2xl font-bold">{{ r.title }}</h1>
          <p class="text-stone-600">{{ r.description }}</p>
        </div>
        <span class="text-sm text-stone-500">{{ r.minutes }} minuten</span>
      </div>

      <div class="mt-6 flex items-center gap-3">
        <span class="text-sm font-medium text-stone-700">Aantal personen</span>
        <button type="button" (click)="change(-1)" class="h-8 w-8 rounded-lg border border-
stone-300 hover:bg-stone-100">-</button>
        <span class="w-6 text-center font-semibold">{{ servings() }}</span>
        <button type="button" (click)="change(1)" class="h-8 w-8 rounded-lg border border-
stone-300 hover:bg-stone-100">+</button>
      </div>

      <h2 class="mt-6 text-lg font-semibold">Ingrediënten</h2>
      <ul class="mt-2">
        @for (item of ingredients(); track $index) {
          <li class="flex justify-between border-b border-stone-100 py-1">
            <span>{{ item.name }}</span>
            <span class="text-stone-600">{{ item.text }}</span>
          </li>
        }
      </ul>

      <h2 class="mt-6 text-lg font-semibold">Bereiding</h2>
      <ol class="mt-2 list-decimal space-y-1 pl-5">
        @for (step of steps(); track $index) {
          <li>{{ step }}</li>
        }
      </ol>
```

```
    </article>

    @if (error(); as message) {
      <p role="alert" class="rounded-lg bg-red-50 px-3 py-2 text-sm text-red-800">{{ message }}</p>
    }

    <div class="flex items-center gap-4">
      <a
        [routerLink]="['/recepten', r.id, 'bewerken']"
        class="rounded-lg border border-stone-300 px-4 py-2 text-sm font-medium text-stone-700
hover:bg-stone-100"
      >
        Bewerken
      </a>
      <button
        type="button"
        (click)="remove(r.id)"
        class="text-sm text-stone-500 underline hover:text-red-700"
      >
        Recept verwijderen
      </button>
    </div>
  </div>
} @else if (loading()) {
  <p class="text-stone-500">Recept laden...</p>
} @else {
  <p class="text-stone-600">
    Dit recept bestaat niet (meer).
    <a routerLink="/recepten" class="text-orange-700 hover:underline">Terug naar alle recepten</a>
  </p>
}
```

Toelichting: `@else if (loading())` toont een laadtekst terwijl de lijst nog binnenkomt. Zonder die tak zou je bij een pagina-verversing op een detailpagina kort "bestaat niet" zien.

recipe-list.ts

Bestand: src/app/recipes/recipe-list/recipe-list.ts

```
import { Component, computed, effect, inject, signal } from '@angular/core';
import { toSignal } from '@angular/core/rxjs-interop';
import { FormControl, ReactiveFormsModule } from '@angular/forms';
import { Router } from '@angular/router';
import { debounceTime, distinctUntilChanged, firstValueFrom, map } from 'rxjs';
import { Panel } from '../../shared/panel/panel';
import { CATEGORIES, Category, sampleRecipes } from '../recipe';
import { RecipeCard } from '../recipe-card/recipe-card';
import { RecipeStore } from '../recipe-store';

const STORAGE_KEY = 'weekmenu:category';

function loadCategory(): Category | 'alle' {
  try {
    const stored = localStorage.getItem(STORAGE_KEY);
    return CATEGORIES.find((category) => category === stored) ?? 'alle';
  } catch {
    return 'alle';
  }
}

@Component({
  selector: 'app-recipe-list',
  imports: [Panel, RecipeCard, ReactiveFormsModule],
  templateUrl: './recipe-list.html',
})
export class RecipeList {
  private readonly store = inject(RecipeStore);
  private readonly router = inject(Router);
```

```
  protected readonly categories = CATEGORIES;
  protected readonly recipes = this.store.recipes;
  protected readonly loading = this.store.loading;
  protected readonly error = this.store.error;
  protected readonly seeding = signal(false);
  protected readonly category = signal<Category | 'alle'>(loadCategory());

  // Reactive Forms: één los invoerveld als FormControl.
  protected readonly search = new FormControl('', { nonNullable: true });

  // RxJS: wacht 250 ms na de laatste toetsaanslag en zet het resultaat om naar een signal.
  private readonly query = toSignal(
    this.search.valueChanges.pipe(
      debounceTime(250),
      map((value) => value.trim().toLowerCase()),
      distinctUntilChanged(),
    ),
    { initialValue: '' },
  );

  protected readonly visible = computed(() => {
    const selected = this.category();
    const query = this.query();

    return this.recipes().filter(
      (recipe) =>
        (selected === 'alle' || recipe.category === selected) &&
        (query === '' || recipe.title.toLowerCase().includes(query)),
    );
  });

  protected readonly chip = 'rounded-full border border-stone-300 px-3 py-1 text-sm capitalize
text-stone-700 hover:bg-stone-100';
  protected readonly chipActive = 'rounded-full border border-orange-600 bg-orange-600 px-3 py-1
text-sm capitalize text-white';

  constructor() {
    // Effect: bewaar de gekozen categorie in de browser (een neveneffect).
    effect(() => {
      try {
        localStorage.setItem(STORAGE_KEY, this.category());
      } catch {
        // opslag niet beschikbaar: negeren
      }
    });
  }

  protected select(category: Category | 'alle'): void {
    this.category.set(category);
  }

  protected open(id: number): void {
    this.router.navigate(['/recepten', id]);
  }

  protected async seed(): Promise<void> {
    this.seeding.set(true);
    try {
      await firstValueFrom(this.store.addMany(sampleRecipes()));
    } finally {
      this.seeding.set(false);
    }
  }
}
```

recipe-list.html

Bestand: src/app/recipes/recipe-list/recipe-list.html

```
<div class="space-y-6">
  <app-panel title="Zoeken en filteren">
```

```
    <span panel-actions class="text-sm text-stone-500">
      {{ visible().length }} van {{ recipes().length }} recepten
    </span>

    <input
      type="search"
      [formControl]="search"
      placeholder="Zoek op titel"
      class="mb-3 w-full rounded-lg border border-stone-300 px-3 py-2 text-sm focus:border-
orange-500 focus:outline-none focus:ring-2 focus:ring-orange-200"
    />

    <div class="flex flex-wrap gap-2">
      <button type="button" (click)="select('alle')" [class]="category() === 'alle' ?
chipActive : chip">
        Alle
      </button>

      @for (item of categories; track item) {
        <button type="button" (click)="select(item)" [class]="category() === item ?
chipActive : chip">
          {{ item }}
        </button>
      }
    </div>
  </app-panel>

  @if (error(); as message) {
    <p role="alert" class="rounded-xl bg-red-50 p-4 text-red-800">{{ message }}</p>
  }

  @if (loading()) {
    <p class="text-stone-500">Recepten laden...</p>
  }

  <div class="grid gap-4 sm:grid-cols-2 lg:grid-cols-3">
    @for (recipe of visible(); track recipe.id) {
      <app-recipe-card [recipe]="recipe" (open)="open($event)" />
    } @empty {
      @if (!loading()) {
        <div class="col-span-full rounded-xl border border-dashed border-stone-300 p-8 text-center
text-stone-500">
          <p>Geen recepten gevonden.</p>

          @if (recipes().length === 0) {
            <button
              type="button"
              (click)="seed()"
              [disabled]="seeding()"
              class="mt-4 rounded-lg bg-orange-600 px-4 py-2 text-sm font-medium text-white
hover:bg-orange-700 disabled:opacity-60"
            >
              {{ seeding() ? 'Bezig...' : 'Voorbeeldrecepten toevoegen' }}
            </button>
          }
        </div>
      }
    }
  </div>
</div>
```

Toelichting

- De lijst toont nu een laadtekst, een foutmelding (uit de store) en, als je nog geen recepten hebt, een knop om de voorbeeldrecepten in **jouw** database te zetten.
- `@empty` bevat een `@if`: blokken mogen geneste control flow bevatten.

Test het hele geheel

1. Ververs de app en klik op "Voorbeeldrecepten toevoegen". Kijk in de Table Editor van Supabase: de zes recepten staan er, met jouw `user_id`.
2. Maak een nieuw recept, bewerk het, verwijder het.
3. Open het netwerktabblad van de browser: bij elk verzoek naar Supabase zie je de headers `apikey` en `Authorization`, door de interceptor toegevoegd.
4. **Test de beveiliging:** registreer een tweede account in een privévenster. Je ziet **geen** recepten van het eerste account. Dat doet de database (RLS), niet je Angular-code.

Deel 9: De weekplanner en de boodschappenlijst

In dit deel bouw je de kern van de app: je verdeelt recepten over de week en Angular rekent de boodschappenlijst voor je uit. Je gebruikt alles wat je tot nu toe geleerd hebt: signals, `computed`, services, `httpResource`, routing en HTTP.

MODULE 27: Het model en de rekenlogica

Doel van deze module

Pure TypeScript-functies voor datums en de boodschappenlijst, zonder Angular. Daardoor zijn ze eenvoudig te testen (Deel 11).

Stap 1: src/app/planner/meal-plan.ts

```
mkdir src\app\planner src\app\shopping
```

Bestand: src/app/planner/meal-plan.ts

```ts
import { Recipe, Unit } from '../recipes/recipe';

export const MEALS = ['lunch', 'diner'] as const;
export type Meal = (typeof MEALS)[number];

// De veldnamen volgen de kolommen in de database (snake_case).
export interface PlanEntry {
  id: number;
  day: string; // yyyy-mm-dd
  meal: Meal;
  recipe_id: number;
}

export interface WeekDay {
  iso: string;
  label: string;
}

export interface ShoppingItem {
  name: string;
  unit: Unit;
  amount: number;
}

// Een datum als yyyy-mm-dd in de lokale tijdzone.
export function toISO(date: Date): string {
  const year = date.getFullYear();
  const month = String(date.getMonth() + 1).padStart(2, '0');
  const day = String(date.getDate()).padStart(2, '0');
  return `${year}-${month}-${day}`;
}

export function addDays(date: Date, days: number): Date {
  return new Date(date.getFullYear(), date.getMonth(), date.getDate() + days);
}
```

```ts
// De maandag van de week waarin de datum valt.
export function startOfWeek(date: Date): Date {
  const offset = (date.getDay() + 6) % 7; // maandag = 0, zondag = 6
  return addDays(date, -offset);
}

export function weekDays(start: Date): WeekDay[] {
  const formatter = new Intl.DateTimeFormat('nl-BE', { weekday: 'long', day: 'numeric', month: 'sho
rt' });

  return Array.from({ length: 7 }, (_, index) => {
    const date = addDays(start, index);
    return { iso: toISO(date), label: formatter.format(date) };
  });
}

// Telt de ingrediënten van alle recepten bij elkaar op (zelfde naam en eenheid = één regel).
export function buildShoppingList(recipes: Recipe[]): ShoppingItem[] {
  const totals = new Map<string, ShoppingItem>();

  for (const recipe of recipes) {
    for (const ingredient of recipe.ingredients) {
      const name = ingredient.name.trim();
      const key = `${name.toLowerCase()}|${ingredient.unit}`;
      const existing = totals.get(key);

      if (existing) {
        existing.amount += ingredient.amount;
      } else {
        totals.set(key, { name, unit: ingredient.unit, amount: ingredient.amount });
      }
    }
  }

  return [...totals.values()]
    .map((item) => ({ ...item, amount: Math.round(item.amount * 100) / 100 }))
    .sort((a, b) => a.name.localeCompare(b.name, 'nl'));
}
```

Toelichting

- `startOfWeek`: `getDay()` geeft zondag = 0 tot zaterdag = 6. `(dag + 6) % 7` maakt er maandag = 0 van, zodat een week op maandag begint (zoals in België).
- `addDays` bouwt een nieuwe `Date` uit jaar, maand en dag: dat is veiliger dan milliseconden optellen, want zomer- en wintertijd veranderen de lengte van een dag.
- `buildShoppingList` gebruikt een `Map` met een **sleutel van naam en eenheid** (`tomaten|g`). Vind je dezelfde sleutel opnieuw, dan tel je de hoeveelheid op. Twee recepten met 400 g tomaten geven 800 g.
- De hoeveelheden ronden we af op twee decimalen om rekenfoutjes met kommagetallen (0.1 + 0.2) te verbergen.

MODULE 28: De PlanStore

Stap 1: Genereren

```
npx ng generate service planner/plan-store --skip-tests
```

Stap 2: plan-store.ts

Bestand: src/app/planner/plan-store.ts

```
import { HttpClient, httpResource } from '@angular/common/http';
import { computed, inject, Service, signal } from '@angular/core';
import { Observable, tap } from 'rxjs';
import { REST_URL } from '../core/api';
import { AuthStore } from '../core/auth-store';
import { addDays, Meal, PlanEntry, startOfWeek, toISO, weekDays } from './meal-plan';

@Service()
export class PlanStore {
  private readonly http = inject(HttpClient);
  private readonly auth = inject(AuthStore);

  // De week die op dit moment getoond wordt (gedeeld door planner en boodschappenlijst).
  readonly weekStart = signal(startOfWeek(new Date()));
  readonly days = computed(() => weekDays(this.weekStart()));

  // De planning van die week: verandert de week, dan wordt automatisch opnieuw opgehaald.
  private readonly list = httpResource<PlanEntry[]>(() => {
    if (!this.auth.isLoggedIn()) return undefined;

    const from = toISO(this.weekStart());
    const to = toISO(addDays(this.weekStart(), 6));

    return {
      url: `${REST_URL}/meal_plan`,
      params: { select: '*', day: [`gte.${from}`, `lte.${to}`] },
    };
  });

  readonly entries = computed(() => (this.list.hasValue() ? this.list.value() : []));
  readonly loading = this.list.isLoading;

  moveWeek(delta: number): void {
    this.weekStart.update((start) => addDays(start, delta * 7));
  }

  thisWeek(): void {
    this.weekStart.set(startOfWeek(new Date()));
  }

  // Zet een recept op een dag en maaltijd. Bestaat die plek al, dan wordt hij overschreven
(upsert).
  set(day: string, meal: Meal, recipeId: number): Observable<unknown> {
    return this.http
      .post(
        `${REST_URL}/meal_plan`,
        { day, meal, recipe_id: recipeId },
        {
          params: { on_conflict: 'user_id,day,meal' },
          headers: { Prefer: 'resolution=merge-duplicates,return=minimal' },
        },
      )
      .pipe(tap(() => this.list.reload()));
  }

  clear(id: number): Observable<unknown> {
    return this.http.delete(`${REST_URL}/meal_plan?id=eq.${id}`).pipe(tap(() => this.list.reload())
);
  }
}
```

Toelichting

- `weekStart` is een **signal** in een service: planner en boodschappenlijst delen dus dezelfde week. Ga je in de planner naar volgende week en open je daarna de boodschappenlijst, dan zie je dezelfde week.

- De `httpResource` leest `weekStart()` in zijn verzoekfunctie. Verandert de week, dan haalt hij **vanzelf** de nieuwe planning op: je hoeft nergens `subscribe` of `switchMap` te schrijven.
- `params: { day: [ 'gte.2026-10-05', 'lte.2026-10-11' ] }` maakt `?day=gte.2026-10-05&day=lte.2026-10-11`: PostgREST leest dat als "day >= … én day <= …".
- Het upsert-gedrag: `on_conflict=user_id,day,meal` met de header `resolution=merge-duplicates` zegt "bestaat deze combinatie al, overschrijf dan het recept". Dat past bij de `unique`-regel uit de database.
- Na een wijziging doen we `reload()`: de lijst wordt opnieuw opgehaald. Tijdens het herladen blijft de oude waarde zichtbaar.

MODULE 29: De pagina Weekplanner

Stap 1: Genereren

```
npx ng generate component planner/planner --skip-tests --style=none
```

Stap 2: planner.ts

Bestand: src/app/planner/planner/planner.ts

```
import { Component, computed, inject, signal } from '@angular/core';
import { RouterLink } from '@angular/router';
import { firstValueFrom } from 'rxjs';
import { RecipeStore } from '../../recipes/recipe-store';
import { MEALS, Meal, PlanEntry } from '../meal-plan';
import { PlanStore } from '../plan-store';

@Component({
  selector: 'app-planner',
  imports: [RouterLink],
  templateUrl: './planner.html',
})
export class Planner {
  private readonly plan = inject(PlanStore);
  private readonly recipeStore = inject(RecipeStore);

  protected readonly recipes = this.recipeStore.recipes;
  protected readonly loading = this.plan.loading;
  protected readonly error = signal<string | null>(null);

  protected readonly range = computed(() => {
    const days = this.plan.days();
    return `${days[0].label} t/m ${days[6].label}`;
  });

  // Per dag twee cellen (lunch en diner), met het bijbehorende plan als dat bestaat.
  protected readonly rows = computed(() => {
    const entries = this.plan.entries();

    return this.plan.days().map((day) => ({
      day,
      cells: MEALS.map((meal) => ({
        meal,
        entry: entries.find((entry) => entry.day === day.iso && entry.meal === meal),
      })),
    }));
  });

  protected readonly select =
    'w-full rounded-lg border border-stone-300 bg-white px-2 py-2 text-sm focus:border-orange-500
```

```
focus:outline-none focus:ring-2 focus:ring-orange-200';

  protected move(delta: number): void {
    this.plan.moveWeek(delta);
  }

  protected today(): void {
    this.plan.thisWeek();
  }

  protected async pick(day: string, meal: Meal, entry: PlanEntry | undefined, event: Event): Promis
e<void> {
    const value = (event.target as HTMLSelectElement).value;
    this.error.set(null);

    try {
      if (value === '') {
        if (entry) await firstValueFrom(this.plan.clear(entry.id));
      } else {
        await firstValueFrom(this.plan.set(day, meal, Number(value)));
      }
    } catch {
      this.error.set('De planning kon niet worden opgeslagen.');
    }
  }
}
```

Stap 3: planner.html

Bestand: src/app/planner/planner/planner.html

```
<div class="space-y-6">
  <div class="flex flex-wrap items-center justify-between gap-3">
    <div>
      <h1 class="text-2xl font-bold">Weekplanner</h1>
      <p class="text-sm capitalize text-stone-500">{{ range() }}</p>
    </div>

    <div class="flex gap-2">
      <button type="button" (click)="move(-1)"
class="rounded-lg border border-stone-300 px-3 py-2 text-sm hover:bg-stone-100">
        Vorige week
      </button>
      <button type="button" (click)="today()" class="rounded-lg border border-stone-300 px-3 py-2
text-sm hover:bg-stone-100">
        Deze week
      </button>
      <button type="button" (click)="move(1)" class="rounded-lg border border-stone-300 px-3 py-2
text-sm hover:bg-stone-100">
        Volgende week
      </button>
    </div>
  </div>

  @if (error(); as message) {
    <p role="alert" class="rounded-lg bg-red-50 px-3 py-2 text-sm text-red-800">{{ message }}</p>
  }

  @if (recipes().length === 0 && !loading()) {
    <p class="rounded-xl border border-dashed border-stone-300 p-8 text-center text-stone-500">
      Je hebt nog geen recepten.
      <a routerLink="/recepten" class="text-orange-700 hover:underline">Voeg er eerst een paar
toe.</a>
    </p>
  } @else {
    <div class="space-y-3">
      @for (row of rows(); track row.day.iso) {
        <div class="grid gap-3 rounded-xl border border-stone-200 bg-white p-4 shadow-sm sm:grid-
cols-[10rem_1fr_1fr] sm:items-center">
          <p class="font-medium capitalize text-stone-800">{{ row.day.label }}</p>
```

```
          @for (cell of row.cells; track cell.meal) {
            <label class="block">
              <span class="mb-1 block text-xs font-medium uppercase tracking-wide text-stone-500">{
{ cell.meal }}</span>
              <select [(change)="pick(row.day.iso, cell.meal, cell.entry, $event)" [class]="sele
ct">
                <option value="" [selected]="!cell.entry">Nog niets gepland</option>
                @for (recipe of recipes(); track recipe.id) {
                  <option [value]="recipe.id" [selected]="cell.entry?.recipe_id === recipe.id">
                    {{ recipe.title }}
                  </option>
                }
              </select>
            </label>
          }
        </div>
      }
    </div>

    <a routerLink="/boodschappen" class="inline-block rounded-lg bg-orange-600 px-4 py-2 text-sm
font-medium text-white hover:bg-orange-700">
      Naar de boodschappenlijst
    </a>
  }
</div>
```

Toelichting

- `rows` is een `computed` die de week combineert met de planning. Verandert de week of de planning, dan rekent hij opnieuw uit; de template is gewoon een lus over het resultaat.
- Een `<select>` met dynamische opties zet je het best met `[selected]` op elke `<option>` in plaats van `[value]` op de `<select>`: de opties bestaan op het moment van binden soms nog niet.
- `(change)="pick(..., $event)"` stuurt het DOM-event mee. In `pick` lees je de gekozen waarde uit het element.
- `@if ... @else`: de volledige planner wordt alleen getoond als er recepten zijn.

MODULE 30: De boodschappenlijst

Stap 1: Genereren

```
npx ng generate component shopping/shopping-list --skip-tests --style=none
```

Stap 2: shopping-list.ts

Bestand: src/app/shopping/shopping-list/shopping-list.ts

```
import { Component, computed, inject, signal } from '@angular/core';
import { RouterLink } from '@angular/router';
import { buildShoppingList, ShoppingItem } from '../../planner/meal-plan';
import { PlanStore } from '../../planner/plan-store';
import { Recipe, formatAmount } from '../../recipes/recipe';
import { RecipeStore } from '../../recipes/recipe-store';

@Component({
  selector: 'app-shopping-list',
  imports: [RouterLink],
  templateUrl: './shopping-list.html',
})
export class ShoppingList {
```

```
  private readonly plan = inject(PlanStore);
  private readonly recipeStore = inject(RecipeStore);

  protected readonly formatAmount = formatAmount;

  protected readonly range = computed(() => {
    const days = this.plan.days();
    return `${days[0].label} t/m ${days[6].label}`;
  });

  // De recepten die deze week gepland staan (een recept dat twee keer gepland is, komt twee keer
voor).
  private readonly planned = computed(() => {
    const byId = new Map(this.recipeStore.recipes().map((recipe) => [recipe.id, recipe] as const));

    return this.plan
      .entries()
      .map((entry) => byId.get(entry.recipe_id))
      .filter((recipe): recipe is Recipe => recipe !== undefined);
  });

  protected readonly items = computed(() => buildShoppingList(this.planned()));
  protected readonly plannedCount = computed(() => this.planned().length);

  // Welke regels zijn afgevinkt (alleen in deze sessie).
  protected readonly checked = signal<ReadonlySet<string>>(new Set());
  protected readonly remaining = computed(
    () => this.items().filter((item) => !this.checked().has(this.key(item))).length,
  );

  protected key(item: ShoppingItem): string {
    return `${item.name.toLowerCase()}|${item.unit}`;
  }

  protected toggle(item: ShoppingItem): void {
    const key = this.key(item);

    this.checked.update((current) => {
      const next = new Set(current);
      if (next.has(key)) next.delete(key);
      else next.add(key);
      return next;
    });
  }
}
```

Stap 3: shopping-list.html

Bestand: src/app/shopping/shopping-list/shopping-list.html

```
<div class="space-y-6">
  <div class="flex flex-wrap items-center justify-between gap-3">
    <div>
      <h1 class="text-2xl font-bold">Boodschappenlijst</h1>
      <p class="text-sm capitalize text-stone-500">{{ range() }}</p>
    </div>
    <a routerLink="/weekplanner" class="text-sm text-orange-700 hover:underline">Terug naar de
planner</a>
  </div>

  @if (items().length === 0) {
    <p class="rounded-xl border border-dashed border-stone-300 p-8 text-center text-stone-500">
      Er staat deze week nog niets gepland.
      <a routerLink="/weekplanner" class="text-orange-700 hover:underline">Plan eerst een paar
maaltijden.</a>
    </p>
  } @else {
    <p class="text-sm text-stone-600">
      {{ plannedCount() }} maaltijden gepland &middot; nog {{ remaining() }} van
{{ items().length }} te kopen
    </p>
```

```
    <ul class="divide-y divide-stone-100 rounded-xl border border-stone-200 bg-white shadow-sm">
      @for (item of items(); track key(item)) {
        <li>
          <label class="flex cursor-pointer items-center gap-3 px-4 py-3">
            <input
              type="checkbox"
              [checked]="checked().has(key(item))"
              (change)="toggle(item)"
              class="h-4 w-4 accent-orange-600"
            />
            <span
              class="flex-1"
              [class.text-stone-400]="checked().has(key(item))"
              [class.line-through]="checked().has(key(item))"
            >
              {{ item.name }}
            </span>
            <span class="text-sm text-stone-600">{{ formatAmount(item.amount, item.unit) }}</span>
          </label>
        </li>
      }
    </ul>
  }
</div>
```

Toelichting

- De boodschappenlijst is **volledig afgeleid**: `planned` zoekt bij elke planning-regel het recept op, `items` telt de ingrediënten op, `remaining` telt wat nog niet afgevinkt is. Wijzig je in de planner iets, dan klopt deze pagina vanzelf.
- `new Map(lijst.map((r) => [r.id, r] as const))` maakt een opzoektabel op id. Het `filter((recipe): recipe is Recipe => ...)` is een **type guard** die TypeScript vertelt dat er daarna geen `undefined` meer in de lijst zit.
- `checked` is een signal met een `Set`. Bij elke wijziging maken we een **nieuwe** `Set` (net als bij arrays): een gewijzigde bestaande set zou Angular niet opmerken.
- `[class.line-through]="voorwaarde"` zet één klasse aan of uit. Voor `track` gebruiken we de sleutel naam+eenheid.
- `protected readonly formatAmount = formatAmount;` maakt een gewone functie beschikbaar in de template.

Stap 4: Routes en navigatie

Bestand: src/app/app.routes.ts

```
import { Routes } from '@angular/router';
import { authGuard, guestGuard } from './core/auth-guard';

export const routes: Routes = [
  {
    path: 'inloggen',
    title: 'Inloggen',
    canActivate: [guestGuard],
    loadComponent: () => import('./auth/login/login').then((m) => m.Login),
  },
  {
    path: '',
    canActivate: [authGuard],
    children: [
      { path: '', pathMatch: 'full', redirectTo: 'recepten' },
      {
        path: 'recepten',
        title: 'Recepten',
```

```
        loadComponent: () => import('./recipes/recipe-list/recipe-list').then((m) => m.RecipeList),
      },
      {
        path: 'recepten/nieuw',
        title: 'Nieuw recept',
        loadComponent: () => import('./recipes/recipe-form/recipe-form').then((m) => m.RecipeForm),
      },
      {
        path: 'recepten/:id/bewerken',
        title: 'Recept bewerken',
        loadComponent: () => import('./recipes/recipe-form/recipe-form').then((m) => m.RecipeForm),
      },
      {
        path: 'recepten/:id',
        title: 'Recept',
        loadComponent: () => import('./recipes/recipe-detail/recipe-detail').then((m) => m.RecipeDe
tail),
      },
      {
        path: 'weekplanner',
        title: 'Weekplanner',
        loadComponent: () => import('./planner/planner/planner').then((m) => m.Planner),
      },
      {
        path: 'boodschappen',
        title: 'Boodschappenlijst',
        loadComponent: () => import('./shopping/shopping-list/shopping-list').then((m) => m.Shoppin
gList),
      },
    ],
  },
  {
    path: '**',
    title: 'Niet gevonden',
    loadComponent: () => import('./shared/not-found/not-found').then((m) => m.NotFound),
  },
];
```

Bestand: src/app/app.html

```
<div class="min-h-screen bg-stone-50">
  <header class="border-b border-stone-200 bg-white">
    <div class="mx-auto flex max-w-5xl flex-wrap items-center justify-between gap-3 px-6 py-4">
      <a routerLink="/recepten" class="text-xl font-bold text-orange-700">Weekmenu</a>

      @if (auth.isLoggedIn()) {
        <nav class="flex flex-wrap items-center gap-1">
          <a
            routerLink="/recepten"
            routerLinkActive="bg-orange-100 text-orange-800"
            [routerLinkActiveOptions]="{ exact: true }"
            [class]="link"
          >
            Recepten
          </a>
          <a routerLink="/recepten/nieuw" routerLinkActive="bg-orange-100 text-orange-800"
[class]="link">
            Nieuw recept
          </a>
          <a routerLink="/weekplanner" routerLinkActive="bg-orange-100 text-orange-800" [class]="l
ink">
            Weekplanner
          </a>
          <a routerLink="/boodschappen" routerLinkActive="bg-orange-100 text-orange-800"
[class]="link">
            Boodschappen
          </a>

          <span class="ml-3 hidden text-sm text-stone-500 lg:inline">{{ auth.user()?.email }}</
span>
          <button type="button" (click)="logout()" [class]="link">Uitloggen</button>
        </nav>
```

```
      }
    </div>
  </header>

  <main class="mx-auto max-w-5xl p-6">
    <router-outlet />
  </main>
</div>
```

Test

1. Plan een paar maaltijden, bijvoorbeeld pasta op maandag en een salade op dinsdag.
2. Open de boodschappenlijst: de ingrediënten zijn opgeteld. Plan hetzelfde recept twee keer en kijk hoe de hoeveelheden verdubbelen.
3. Ga naar de volgende week: de planning is leeg, de lijst ook.

Oefeningen

1. Houd rekening met het aantal personen per maaltijd: voeg een kolom `servings` toe aan `meal_plan` en schaal de ingrediënten in de boodschappenlijst met `scaleAmount`.
2. Laat de boodschappenlijst onthouden wat afgevinkt is, bijvoorbeeld met een `effect` en `localStorage` per week.
3. Voeg een knop "Hele dag leegmaken" toe aan de planner.

Deel 10: Pipes, directives en @defer

MODULE 31: Pipes

Een **pipe** zet een waarde in de template om naar een andere weergave: `{{ waarde | pipenaam }}`. Het is het Angular-equivalent van een *filter* (Vue 2) of een kleine helperfunctie.

Ingebouwde pipes

Pipe	Voorbeeld	Resultaat
`date`	`{{ vandaag | date: 'dd/MM/yyyy' }}`	`06/10/2026`
`currency`	`{{ 4.5 | currency: 'EUR' }}`	`€4.50`
`number`	`{{ 3.14159 | number: '1.0-2' }}`	`3.14`
`uppercase`, `lowercase`, `titlecase`	`{{ 'soep' | titlecase }}`	`Soep`
`slice`	`{{ lijst | slice: 0 : 3 }}`	de eerste drie
`json`	`{{ object | json }}`	handig om te debuggen
`async`	`{{ observable$ | async }}`	abonneert en ontabonneert vanzelf

Een eigen pipe

We tonen bereidingstijden als "1 u 15 min" in plaats van "75 min".

```
npx ng generate pipe shared/duration --skip-tests
```

Bestand: src/app/shared/duration-pipe.ts

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'duration',
})
export class DurationPipe implements PipeTransform {
  transform(minutes: number): string {
    if (minutes < 60) return `${minutes} min`;

    const hours = Math.floor(minutes / 60);
    const rest = minutes % 60;

    return rest === 0 ? `${hours} u` : `${hours} u ${rest} min`;
  }
}
```

Gebruik hem in de receptkaart. Importeer de pipe in het component en pas de template aan:

Bestand: src/app/recipes/recipe-card/recipe-card.ts

```
import { Component, input, output } from '@angular/core';
import { DurationPipe } from '../../shared/duration-pipe';
import { Recipe } from '../recipe';

@Component({
  selector: 'app-recipe-card',
  imports: [DurationPipe],
  templateUrl: './recipe-card.html',
})
export class RecipeCard {
  readonly recipe = input.required<Recipe>();
  readonly open = output<number>();
}
```

Bestand: src/app/recipes/recipe-card/recipe-card.html

```
@let r = recipe();

<article class="flex h-full flex-col rounded-xl border border-stone-200 bg-white p-4 shadow-sm">
  <div class="flex items-start justify-between gap-3">
    <h3 class="font-semibold text-stone-900">{{ r.title }}</h3>

    @switch (r.category) {
      @case ('ontbijt') {
        <span class="rounded-full bg-amber-100 px-2.5 py-1 text-xs font-medium text-amber-800">Ontb
ijt</span>
      }
      @case ('lunch') {
        <span class="rounded-full bg-sky-100 px-2.5 py-1 text-xs font-medium text-sky-800">Lunch</s
pan>
      }
      @case ('diner') {
        <span class="rounded-full bg-orange-100 px-2.5 py-1 text-xs font-medium text-orange-800">Di
ner</span>
      }
      @default {
        <span class="rounded-full bg-pink-100 px-2.5 py-1 text-xs font-medium text-pink-800">Desser
t</span>
      }
    }
  </div>

  <p class="mt-1 text-sm text-stone-600">{{ r.description }}</p>

  <p class="mt-3 text-sm text-stone-500">
    {{ r.minutes | duration }} &middot; {{ r.servings }} personen &middot;
{{ r.ingredients.length }} ingrediënten
  </p>

  <button
    type="button"
    (click)="open.emit(r.id)"
    class="mt-4 rounded-lg border border-stone-300 px-3 py-2 text-sm font-medium text-stone-700
hover:bg-stone-100"
  >
    Bekijk recept
  </button>
</article>
```

Toelichting

- Een pipe is een klasse met `@Pipe({ name })` en een methode `transform`. De eerste parameter is de waarde links van de `|`; extra parameters geef je met `:` mee (`{{ x | pipe: a : b }}`).
- Pipes zijn standaard **puur**: Angular voert ze alleen opnieuw uit als de invoer verandert. Dat maakt ze goedkoop.

- Een pipe moet in de `imports` van het component staan dat hem gebruikt.

MODULE 32: Directives

Een **directive** voegt gedrag toe aan een bestaand element, zonder eigen template. Er zijn drie soorten:

Soort	Wat doet het?	Voorbeeld
Component	een directive met een template	`<app-recipe-card>`
Attribuut-directive	verandert uiterlijk of gedrag van een element	`routerLink`, `[formField]`
Structurele directive	voegt elementen toe of verwijdert ze	oud: `*ngIf`, `*ngFor`; nu `@if`, `@for`

Een eigen attribuut-directive: kopieer naar het klembord

We maken een directive waarmee je tekst kopieert door op een knop te klikken, en voegen een knop "Kopieer lijst" toe aan de boodschappenlijst.

```
npx ng generate directive shared/copy --skip-tests
```

Bestand: src/app/shared/copy.ts

```ts
import { Directive, input, signal } from '@angular/core';

@Directive({
  selector: '[appCopy]',
  exportAs: 'appCopy',
  host: {
    '(click)': 'copy()',
  },
})
export class Copy {
  // De tekst die gekopieerd moet worden.
  readonly appCopy = input.required<string>();

  // Is waar tot anderhalve seconde na het kopiëren.
  readonly copied = signal(false);

  async copy(): Promise<void> {
    await navigator.clipboard.writeText(this.appCopy());

    this.copied.set(true);
    setTimeout(() => this.copied.set(false), 1500);
  }
}
```

Gebruik hem in de boodschappenlijst. Voeg de directive toe aan `imports`, maak een tekstversie van de lijst en plaats de knop:

Bestand: src/app/shopping/shopping-list/shopping-list.ts

```ts
import { Component, computed, inject, signal } from '@angular/core';
import { RouterLink } from '@angular/router';
import { buildShoppingList, ShoppingItem } from '../../planner/meal-plan';
import { PlanStore } from '../../planner/plan-store';
import { Recipe, formatAmount } from '../../recipes/recipe';
import { RecipeStore } from '../../recipes/recipe-store';
import { Copy } from '../../shared/copy';
```

```ts
@Component({
  selector: 'app-shopping-list',
  imports: [RouterLink, Copy],
  templateUrl: './shopping-list.html',
})
export class ShoppingList {
  private readonly plan = inject(PlanStore);
  private readonly recipeStore = inject(RecipeStore);

  protected readonly formatAmount = formatAmount;

  protected readonly range = computed(() => {
    const days = this.plan.days();
    return `${days[0].label} t/m ${days[6].label}`;
  });

  // De recepten die deze week gepland staan (een recept dat twee keer gepland is, komt twee keer voor).
  private readonly planned = computed(() => {
    const byId = new Map(this.recipeStore.recipes().map((recipe) => [recipe.id, recipe] as const));

    return this.plan
      .entries()
      .map((entry) => byId.get(entry.recipe_id))
      .filter((recipe): recipe is Recipe => recipe !== undefined);
  });

  protected readonly items = computed(() => buildShoppingList(this.planned()));
  protected readonly plannedCount = computed(() => this.planned().length);

  // De lijst als gewone tekst, om te kopiëren of te delen.
  protected readonly asText = computed(() =>
    this.items()
      .map((item) => `${formatAmount(item.amount, item.unit)} ${item.name}`)
      .join('\n'),
  );

  // Welke regels zijn afgevinkt (alleen in deze sessie).
  protected readonly checked = signal<ReadonlySet<string>>(new Set());
  protected readonly remaining = computed(
    () => this.items().filter((item) => !this.checked().has(this.key(item))).length,
  );

  protected key(item: ShoppingItem): string {
    return `${item.name.toLowerCase()}|${item.unit}`;
  }

  protected toggle(item: ShoppingItem): void {
    const key = this.key(item);

    this.checked.update((current) => {
      const next = new Set(current);
      if (next.has(key)) next.delete(key);
      else next.add(key);
      return next;
    });
  }
}
```

Bestand: src/app/shopping/shopping-list/shopping-list.html

```html
<div class="space-y-6">
  <div class="flex flex-wrap items-center justify-between gap-3">
    <div>
      <h1 class="text-2xl font-bold">Boodschappenlijst</h1>
      <p class="text-sm capitalize text-stone-500">{{ range() }}</p>
    </div>
    <a routerLink="/weekplanner" class="text-sm text-orange-700 hover:underline">Terug naar de planner</a>
  </div>
```

```
  @if (items().length === 0) {
    <p class="rounded-xl border border-dashed border-stone-300 p-8 text-center text-stone-500">
      Er staat deze week nog niets gepland.
      <a routerLink="/weekplanner" class="text-orange-700 hover:underline">Plan eerst een paar
maaltijden.</a>
    </p>
  } @else {
    <div class="flex flex-wrap items-center justify-between gap-3">
      <p class="text-sm text-stone-600">
        {{ plannedCount() }} maaltijden gepland &middot; nog {{ remaining() }} van
{{ items().length }} te kopen
      </p>

      <button
        type="button"
        [appCopy]="asText()"
        #copier="appCopy"
        class="rounded-lg border border-stone-300 px-3 py-2 text-sm hover:bg-stone-100"
      >
        {{ copier.copied() ? 'Gekopieerd!' : 'Kopieer lijst' }}
      </button>
    </div>

    <ul class="divide-y divide-stone-100 rounded-xl border border-stone-200 bg-white shadow-sm">
      @for (item of items(); track key(item)) {
        <li>
          <label class="flex cursor-pointer items-center gap-3 px-4 py-3">
            <input
              type="checkbox"
              [checked]="checked().has(key(item))"
              (change)="toggle(item)"
              class="h-4 w-4 accent-orange-600"
            />
            <span
              class="flex-1"
              [class.text-stone-400]="checked().has(key(item))"
              [class.line-through]="checked().has(key(item))"
            >
              {{ item.name }}
            </span>
            <span class="text-sm text-stone-600">{{ formatAmount(item.amount, item.unit) }}</span>
          </label>
        </li>
      }
    </ul>
  }
</div>
```

Toelichting

- `@Directive({ selector: '[appCopy]' })`: de vierkante haken betekenen "elk element met dit attribuut". Gebruik een eigen voorvoegsel (`app`) zodat je niet botst met HTML of bibliotheken.
- `host: { '(click)': 'copy()' }` koppelt een gebeurtenis van het **element waarop de directive staat** aan een methode. Zo luisteren directives naar hun host. (In oudere code gebruik je `@HostListener('click')` en `@HostBinding`; zie Bijlage A.)
- `appCopy = input.required<string>()`: de directive kan een input hebben met dezelfde naam als zijn selector. Daardoor schrijf je `[appCopy]="asText()"`.
- `exportAs: 'appCopy'` laat je de directive in de template **ophalen** met een template-referentie: `#copier="appCopy"`. `copier.copied()` leest daarna de signal van de directive, om de knoptekst te veranderen.
- `navigator.clipboard.writeText` werkt alleen in een veilige context (`https` of `localhost`).

MODULE 33: @defer: onderdelen later laden

Doel van deze module

Een zwaar of weinig gebruikt onderdeel van een pagina pas laden als het nodig is. Dat versnelt het eerste scherm.

Met `@defer` markeer je een blok van de template dat Angular als **apart bestand** bouwt en pas laadt op een gekozen moment.

Trigger	Wanneer wordt geladen?
`on idle` (standaard)	zodra de browser niets te doen heeft
`on viewport`	zodra het blok (of de placeholder) in beeld komt
`on interaction`	bij een klik of toets op de placeholder
`on hover`	bij hoveren
`on timer(2s)`	na een vertraging
`when voorwaarde`	zodra een expressie waar wordt

Er horen drie optionele blokken bij: `@placeholder` (getoond voor het laden begint), `@loading` (tijdens het laden) en `@error` (als het mislukt).

Stap 1: Een eigen component voor de bereiding

```
npx ng generate component recipes/recipe-steps --skip-tests --style=none
```

Bestand: src/app/recipes/recipe-steps/recipe-steps.ts

```
import { Component, input } from '@angular/core';

@Component({
  selector: 'app-recipe-steps',
  template: `
    <ol class="mt-2 list-decimal space-y-1 pl-5">
      @for (step of steps(); track $index) {
        <li>{{ step }}</li>
      }
    </ol>
  `,
})
export class RecipeSteps {
  readonly steps = input.required<string[]>();
}
```

Het bestand `recipe-steps.html` dat de CLI maakte, mag je verwijderen:

```
del src\app\recipes\recipe-steps\recipe-steps.html
```

Stap 2: De detailpagina

Importeer het component en vervang de bereiding in de template door een `@defer`-blok:

Bestand: src/app/recipes/recipe-detail/recipe-detail.ts

```
import { Component, computed, inject, input, linkedSignal, signal } from '@angular/core';
import { Router, RouterLink } from '@angular/router';
import { firstValueFrom } from 'rxjs';
import { formatAmount, scaleAmount } from '../recipe';
import { RecipeStore } from '../recipe-store';
import { RecipeSteps } from '../recipe-steps/recipe-steps';

@Component({
  selector: 'app-recipe-detail',
  imports: [RouterLink, RecipeSteps],
  templateUrl: './recipe-detail.html',
})
export class RecipeDetail {
  private readonly store = inject(RecipeStore);
  private readonly router = inject(Router);

  // Komt uit de URL: /recepten/:id
  readonly id = input.required<string>();

  protected readonly loading = this.store.loading;
  protected readonly error = signal<string | null>(null);

  protected readonly recipe = computed(() => this.store.byId(Number(this.id())));

  // Begint bij het aantal personen van het recept en wordt opnieuw ingesteld als het recept
verandert.
  protected readonly servings = linkedSignal(() => this.recipe()?.servings ?? 1);

  protected readonly ingredients = computed(() => {
    const recipe = this.recipe();
    if (!recipe) return [];

    return recipe.ingredients.map((ingredient) => ({
      name: ingredient.name,
      text: formatAmount(scaleAmount(ingredient.amount, recipe.servings, this.servings()), ingredie
nt.unit),
    }));
  });

  protected readonly steps = computed(
    () => this.recipe()?.steps.split('\n').filter((line) => line.trim() !== '') ?? [],
  );

  protected change(delta: number): void {
    this.servings.update((value) => Math.min(24, Math.max(1, value + delta)));
  }

  protected async remove(id: number): Promise<void> {
    this.error.set(null);

    try {
      await firstValueFrom(this.store.remove(id));
      await this.router.navigateByUrl('/recepten');
    } catch {
      this.error.set('Het recept kon niet verwijderd worden.');
    }
  }
}
```

Bestand: src/app/recipes/recipe-detail/recipe-detail.html

```
@if (recipe(); as r) {
  <div class="space-y-6">
    <a routerLink="/recepten" class="text-sm text-orange-700 hover:underline">Terug naar alle
recepten</a>

    <article class="rounded-xl border border-stone-200 bg-white p-6 shadow-sm">
      <div class="flex flex-wrap items-start justify-between gap-3">
        <div>
          <h1 class="text-2xl font-bold">{{ r.title }}</h1>
```

```
          <p class="text-stone-600">{{ r.description }}</p>
        </div>
        <span class="text-sm text-stone-500">{{ r.minutes }} minuten</span>
      </div>

      <div class="mt-6 flex items-center gap-3">
        <span class="text-sm font-medium text-stone-700">Aantal personen</span>
        <button type="button" (click)="change(-1)" class="h-8 w-8 rounded-lg border border-
stone-300 hover:bg-stone-100">-</button>
        <span class="w-6 text-center font-semibold">{{ servings() }}</span>
        <button type="button" (click)="change(1)" class="h-8 w-8 rounded-lg border border-
stone-300 hover:bg-stone-100">+</button>
      </div>

      <h2 class="mt-6 text-lg font-semibold">Ingrediënten</h2>
      <ul class="mt-2">
        @for (item of ingredients(); track $index) {
          <li class="flex justify-between border-b border-stone-100 py-1">
            <span>{{ item.name }}</span>
            <span class="text-stone-600">{{ item.text }}</span>
          </li>
        }
      </ul>

      <h2 class="mt-6 text-lg font-semibold">Bereiding</h2>
      @defer (on viewport) {
        <app-recipe-steps [steps]="steps()" />
      } @placeholder {
        <p class="mt-2 text-sm text-stone-400">De bereiding verschijnt zodra je hier komt...</p>
      } @loading (minimum 200ms) {
        <p class="mt-2 text-sm text-stone-400">Laden...</p>
      }
    </article>

    @if (error(); as message) {
      <p role="alert" class="rounded-lg bg-red-50 px-3 py-2 text-sm text-red-800">{{ message }}</p>
    }

    <div class="flex items-center gap-4">
      <a
        [routerLink]="['/recepten', r.id, 'bewerken']"
        class="rounded-lg border border-stone-300 px-4 py-2 text-sm font-medium text-stone-700
hover:bg-stone-100"
      >
        Bewerken
      </a>
      <button
        type="button"
        (click)="remove(r.id)"
        class="text-sm text-stone-500 underline hover:text-red-700"
      >
        Recept verwijderen
      </button>
    </div>
  </div>
} @else if (loading()) {
  <p class="text-stone-500">Recept laden...</p>
} @else {
  <p class="text-stone-600">
    Dit recept bestaat niet (meer).
    <a routerLink="/recepten" class="text-orange-700 hover:underline">Terug naar alle recepten</a>
  </p>
}
```

Toelichting

- `@defer (on viewport) { ... }`: de inhoud (en de code van `RecipeSteps`) wordt pas geladen als de placeholder in beeld komt. Angular maakt van het uitgestelde component bij het bouwen automatisch een **apart bestand**.

- Voor `on viewport` is de `@placeholder` het element dat Angular in de gaten houdt. Daarom moet de placeholder één element met een weergave zijn.
- `@loading (minimum 200ms)` zorgt dat de laadtekst minstens 200 ms zichtbaar blijft, zodat er geen flits ontstaat.
- Alleen **standalone** componenten die niet elders al in dezelfde template gebruikt worden, kunnen echt apart geladen worden. Dat is hier het geval.
- Open het netwerktabblad van je browser en kijk: het bestand voor `RecipeSteps` laadt pas na het scrollen naar de bereiding (bij een korte pagina meteen).

> **Vergelijking met Vue/React:** `@defer` is `defineAsyncComponent` (Vue) of `React.lazy` met `Suspense` (React), maar rechtstreeks in de template met ingebouwde triggers.

MODULE 34: Levenscyclus en template-referenties

Levenscyclus: wanneer draait wat?

Een component wordt aangemaakt, getekend en vernietigd. In moderne Angular heb je zelden hooks nodig, omdat signals het werk doen:

Ik wil…	Gebruik
iets doen bij het aanmaken	de `constructor` of een veldinitialisatie
iets doen **nadat** het component de eerste keer getekend is (DOM beschikbaar)	`afterNextRender(() => { ... })`
iets doen bij **elke** render	`afterEveryRender(...)`
opruimen als het component verdwijnt	`inject(DestroyRef).onDestroy(() => ...)`
reageren op een input die verandert	`computed`, `linkedSignal` of `effect`

Oudere code gebruikt klassemethoden zoals `ngOnInit`, `ngOnChanges`, `ngAfterViewInit` en `ngOnDestroy`. Die werken nog steeds en komen in vrijwel elk bestaand project voor (Bijlage A). De signal-aanpak is eenvoudiger omdat de afhankelijkheden vanzelf bijgehouden worden.

Template-referenties

Met `#naam` geef je een element of component een **naam in de template**:

```
<input #zoek type="search" />
<button type="button" (click)="zoek.focus()">Zoek focussen</button>
```

- `#zoek` is een verwijzing naar het DOM-element; je gebruikt hem direct in de template.
- Met `#copier="appCopy"` (Module 32) verwijs je naar een directive.
- Vanuit de klasse haal je ze op met `viewChild('zoek')` of `viewChild(Copy)`: een **signal**, `viewChild.required<ElementRef>('zoek')`, die beschikbaar is na het eerste renderen.

Oefeningen

1. Maak een pipe `plural` die `{{ 3 | plural: 'recept' : 'recepten' }}` omzet in `recepten`.
2. Voeg een directive `appAutofocus` toe met `afterNextRender` en `inject(ElementRef)`, en gebruik hem op het e-mailveld van de loginpagina.
3. Zet het formulier voor het ingrediënt in een `@defer (on interaction)`.

Deel 11: Testen met Vitest

MODULE 35: Hoe test je Angular?

Nieuwe Angular-projecten gebruiken **Vitest** als testrunner. Je voert de tests uit met:

```
npx ng test
```

Dat start de tests in "watch"-modus: bij elke wijziging draaien ze opnieuw. Voor een eenmalige run (bijvoorbeeld in een build-pipeline):

```
npx ng test --no-watch
```

Een testbestand heet `iets.spec.ts` en staat naast het bestand dat het test. De functies `describe` (groep), `it` (één test) en `expect` (controle) zijn overal beschikbaar.

Wat test je?

Soort	Voorbeeld	Moeilijkheid
Pure functies	`scaleAmount`, `buildShoppingList`	eenvoudig, geen Angular nodig
Pipes	`DurationPipe`	eenvoudig
Componenten	`RecipeCard` toont een titel en stuurt een event	gemiddeld, met `TestBed`
Services met HTTP	`RecipeStore` haalt recepten op	gemiddeld, met `HttpTestingController`
Guards	`authGuard` stuurt door naar het inloggen	gemiddeld

Je hoeft niet alles te testen. Begin bij de logica waar fouten het duurst zijn: berekeningen en data-omzettingen.

MODULE 36: Pure functies en pipes testen

src/app/recipes/recipe.spec.ts

Bestand: src/app/recipes/recipe.spec.ts

```
import { formatAmount, scaleAmount } from './recipe';

describe('scaleAmount', () => {
  it('rekent 250 g voor 4 personen om naar 125 g voor 2 personen', () => {
    expect(scaleAmount(250, 4, 2)).toBe(125);
  });

  it('rondt af op twee decimalen', () => {
    expect(scaleAmount(100, 3, 1)).toBe(33.33);
  });
```

```
  it('geeft dezelfde hoeveelheid terug bij hetzelfde aantal personen', () => {
    expect(scaleAmount(40, 4, 4)).toBe(40);
  });
});

describe('formatAmount', () => {
  it('schrijft stuks in enkelvoud en meervoud', () => {
    expect(formatAmount(1, 'stuk')).toBe('1 stuk');
    expect(formatAmount(2, 'stuk')).toBe('2 stuks');
  });

  it('gebruikt een komma als decimaalteken', () => {
    expect(formatAmount(2.5, 'el')).toBe('2,5 el');
  });
});
```

src/app/planner/meal-plan.spec.ts

Bestand: src/app/planner/meal-plan.spec.ts

```
import { Recipe } from '../recipes/recipe';
import { buildShoppingList, startOfWeek, toISO, weekDays } from './meal-plan';

function recipe(id: number, ingredients: Recipe['ingredients']): Recipe {
  return { id, title: `Recept ${id}`, description: '', category: 'diner', servings: 4, minutes:
30, ingredients, steps: '' };
}

describe('startOfWeek', () => {
  it('geeft de maandag van die week', () => {
    // 7 oktober 2026 is een woensdag
    expect(toISO(startOfWeek(new Date(2026, 9, 7)))).toBe('2026-10-05');
  });

  it('laat een maandag ongewijzigd', () => {
    expect(toISO(startOfWeek(new Date(2026, 9, 5)))).toBe('2026-10-05');
  });

  it('rekent zondag nog bij de voorgaande week', () => {
    expect(toISO(startOfWeek(new Date(2026, 9, 11)))).toBe('2026-10-05');
  });
});

describe('weekDays', () => {
  it('geeft zeven opeenvolgende dagen', () => {
    const days = weekDays(new Date(2026, 9, 5));

    expect(days).toHaveLength(7);
    expect(days[0].iso).toBe('2026-10-05');
    expect(days[6].iso).toBe('2026-10-11');
  });
});

describe('buildShoppingList', () => {
  it('telt dezelfde ingrediënten bij elkaar op', () => {
    const list = buildShoppingList([
      recipe(1, [{ name: 'Tomaten', amount: 400, unit: 'g' }]),
      recipe(2, [{ name: 'tomaten', amount: 300, unit: 'g' }]),
    ]);

    expect(list).toEqual([{ name: 'Tomaten', unit: 'g', amount: 700 }]);
  });

  it('houdt verschillende eenheden gescheiden', () => {
    const list = buildShoppingList([
      recipe(1, [
        { name: 'Olijfolie', amount: 2, unit: 'el' },
        { name: 'Olijfolie', amount: 100, unit: 'ml' },
      ]),
    ]);
```

```
    expect(list).toHaveLength(2);
  });

  it('sorteert alfabetisch en geeft een lege lijst voor geen recepten', () => {
    expect(buildShoppingList([])).toEqual([]);

    const list = buildShoppingList([
      recipe(1, [
        { name: 'Zout', amount: 1, unit: 'tl' },
        { name: 'Bloem', amount: 100, unit: 'g' },
      ]),
    ]);

    expect(list.map((item) => item.name)).toEqual(['Bloem', 'Zout']);
  });
});
```

src/app/shared/duration-pipe.spec.ts

Bestand: src/app/shared/duration-pipe.spec.ts

```
import { DurationPipe } from './duration-pipe';

describe('DurationPipe', () => {
  const pipe = new DurationPipe();

  it('toont minuten onder het uur', () => {
    expect(pipe.transform(45)).toBe('45 min');
  });

  it('toont hele uren', () => {
    expect(pipe.transform(60)).toBe('1 u');
  });

  it('toont uren en minuten', () => {
    expect(pipe.transform(75)).toBe('1 u 15 min');
  });
});
```

Toelichting

- Een pure functie test je door hem aan te roepen en het resultaat te vergelijken: `expect(waarde).toBe(verwacht)`.
- `toBe` vergelijkt met `===` (getallen, teksten). `toEqual` vergelijkt de **inhoud** van objecten en arrays. `toHaveLength` controleert de lengte.
- De pipe is een gewone klasse: je maakt er met `new` een aan, zonder Angular.
- Een goede test beschrijft in de naam **wat** er verwacht wordt, zodat een mislukte test meteen duidelijk is.
- De laatste drie tests van `startOfWeek` testen **randgevallen** (maandag, zondag). Fouten zitten bijna altijd in de randen.

MODULE 37: Een component en een service testen

Een component testen met TestBed

Bestand: src/app/recipes/recipe-card/recipe-card.spec.ts

```
import { TestBed } from '@angular/core/testing';
import { SAMPLE_RECIPES } from '../recipe';
import { RecipeCard } from './recipe-card';

describe('RecipeCard', () => {
  async function render() {
    const fixture = TestBed.createComponent(RecipeCard);
    fixture.componentRef.setInput('recipe', SAMPLE_RECIPES[0]);
    await fixture.whenStable();
    return fixture;
  }

  it('toont de titel en de bereidingstijd', async () => {
    const fixture = await render();
    const text = fixture.nativeElement.textContent as string;

    expect(text).toContain('Pannenkoeken');
    expect(text).toContain('25 min');
  });

  it('stuurt het id mee bij een klik op de knop', async () => {
    const fixture = await render();
    const opened: number[] = [];
    fixture.componentInstance.open.subscribe((id) => opened.push(id));

    fixture.nativeElement.querySelector('button').click();

    expect(opened).toEqual([1]);
  });
});
```

Toelichting

- `TestBed.createComponent(...)` maakt het component zoals Angular dat zou doen, in een kleine testomgeving.
- `fixture.componentRef.setInput('recipe', ...)` zet een `input()`. Je kunt een signal-input niet rechtstreeks toewijzen.
- `await fixture.whenStable()` wacht tot Angular klaar is met tekenen.
- `fixture.nativeElement` is het echte DOM-element. Je test wat de gebruiker ziet (`textContent`) en doet (`click()`), niet de interne signals.

Een service met HTTP testen

Bestand: src/app/recipes/recipe-store.spec.ts

```
import { provideHttpClient } from '@angular/common/http';
import { HttpTestingController, provideHttpClientTesting } from '@angular/common/http/testing';
import { ApplicationRef, signal } from '@angular/core';
import { TestBed } from '@angular/core/testing';
import { AuthStore } from '../core/auth-store';
import { SAMPLE_RECIPES } from './recipe';
import { RecipeStore } from './recipe-store';

describe('RecipeStore', () => {
  function setup(loggedIn: boolean) {
    TestBed.configureTestingModule({
```

```
      providers: [
        provideHttpClient(),
        provideHttpClientTesting(),
        // Een nep-AuthStore: de echte zou Supabase opstarten.
        { provide: AuthStore, useValue: { isLoggedIn: signal(loggedIn) } },
      ],
    });

    return { store: TestBed.inject(RecipeStore), http: TestBed.inject(HttpTestingController) };
  }

  it('haalt niets op zolang je niet ingelogd bent', () => {
    const { http } = setup(false);
    TestBed.tick();

    http.expectNone(() => true);
  });

  it('haalt recepten op en sorteert ze op titel', async () => {
    const { store, http } = setup(true);
    TestBed.tick();

    http.expectOne((request) => request.url.endsWith('/rest/v1/recipes')).flush(SAMPLE_RECIPES);
    await TestBed.inject(ApplicationRef).whenStable();

    expect(store.recipes()).toHaveLength(6);
    expect(store.recipes()[0].title).toBe('Chocolademousse');
  });

  it('zoekt een recept op id', async () => {
    const { store, http } = setup(true);
    TestBed.tick();

    http.expectOne((request) => request.url.endsWith('/rest/v1/recipes')).flush(SAMPLE_RECIPES);
    await TestBed.inject(ApplicationRef).whenStable();

    expect(store.byId(3)?.title).toBe('Spaghetti bolognese');
    expect(store.byId(999)).toBeUndefined();
  });
});
```

Toelichting

- `provideHttpClientTesting()` vervangt het echte netwerk door een **nep-server**. Via `HttpTestingController` bepaal jij welke verzoeken er verwacht worden en wat het antwoord is.
- `{ provide: AuthStore, useValue: { ... } }` **vervangt een service** door een eenvoudig object. Dit is waarom dependency injection nuttig is: je test de store zonder Supabase.
- `TestBed.tick()` laat Angular zijn effecten en resources uitvoeren; pas dan start de `httpResource` zijn verzoek.
- `expectOne(...).flush(data)` controleert dat er precies één passend verzoek was en beantwoordt het met `data`.
- `expectNone` controleert dat er **geen** verzoek is gedaan (hier: niet ingelogd).

Een guard testen

Bestand: src/app/core/auth-guard.spec.ts

```
import { signal } from '@angular/core';
import { TestBed } from '@angular/core/testing';
import { ActivatedRouteSnapshot, provideRouter, RouterStateSnapshot, UrlTree } from '@angular/
router';
import { authGuard, guestGuard } from './auth-guard';
import { AuthStore } from './auth-store';

describe('guards', () => {
```

```
  const route = {} as ActivatedRouteSnapshot;
  const state = { url: '/weekplanner' } as RouterStateSnapshot;

  function setup(loggedIn: boolean) {
    TestBed.configureTestingModule({
      providers: [
        provideRouter([]),
        { provide: AuthStore, useValue: { whenReady: () => Promise.resolve(), isLoggedIn: signal(lo
ggedIn) } },
      ],
    });
  }

  it('authGuard laat een ingelogde gebruiker door', async () => {
    setup(true);
    const result = await TestBed.runInInjectionContext(() => authGuard(route, state));

    expect(result).toBe(true);
  });

  it('authGuard stuurt een bezoeker naar het inloggen en onthoudt de pagina', async () => {
    setup(false);
    const result = await TestBed.runInInjectionContext(() => authGuard(route, state));

    expect(result).toBeInstanceOf(UrlTree);
    expect(result.toString()).toBe('/inloggen?terug=%2Fweekplanner');
  });

  it('guestGuard stuurt een ingelogde gebruiker weg van de loginpagina', async () => {
    setup(true);
    const result = await TestBed.runInInjectionContext(() => guestGuard(route, state));

    expect(result).toBeInstanceOf(UrlTree);
  });
});
```

Toelichting: een guard is een gewone functie die `inject()` gebruikt. `TestBed.runInInjectionContext(...)` voert hem uit in een omgeving waar `inject` werkt, met onze nep-`AuthStore`.

De tests draaien

```
npx ng test --no-watch
```

Je ziet een lijst met geslaagde tests. Breek een test expres (verander `125` in `124`) en kijk hoe de melding eruitziet.

Oefeningen

1. Schrijf een test voor `RecipeDetail`: controleer dat het aantal personen terug naar het basisaantal gaat als het recept verandert (`linkedSignal`).
2. Schrijf een test die controleert dat `ShoppingList` een lege-lijst-melding toont als er niets gepland is.
3. Voeg een test toe voor `PlanStore.set` met `HttpTestingController`: controleer de URL, de `Prefer`-header en de body.

Deel 12: Bouwen, SSR en deployen

MODULE 38: De productiebuild

```
npm run build
```

Dit voert `ng build` uit en maakt de map `dist/weekmenu/browser` met alles wat je op een server zet: `index.html`, JavaScript en CSS.

Wat gebeurt er bij een productiebuild?

- De code wordt **geminimaliseerd** en ongebruikte code wordt verwijderd (*tree shaking*).
- De bestanden krijgen een **hash** in hun naam, zodat browsers ze veilig kunnen cachen.
- Elke `loadComponent` en `@defer` wordt een **apart bestand** (*code splitting*). In de uitvoer zie je een tabel met "Initial chunk files" (wat meteen geladen wordt) en "Lazy chunk files" (wat later geladen wordt).
- De **budgets** in `angular.json` waarschuwen als je app te groot wordt: standaard een waarschuwing vanaf 500 kB voor de eerste download en een fout vanaf 1 MB.
- Het bestand `environment.ts` wordt gebruikt (bij een ontwikkelbuild vervangt Angular het door `environment.development.ts`).

De waarschuwing over de grootte

Je ziet bij deze app waarschijnlijk een waarschuwing: *"bundle initial exceeded maximum budget"* (bij ons 543 kB tegenover het budget van 500 kB). De bibliotheek `@supabase/supabase-js` is groot en zit in de eerste download. Dat is geen fout: het is een bewust ingebouwde alarmbel.

Je hebt twee opties:

1. **Het budget aanpassen** als je de grootte acceptabel vindt. Zoek in `angular.json` de sectie `budgets` bij de productieconfiguratie en verhoog `maximumWarning` van `500kB` naar bijvoorbeeld `700kB`.
2. **De grootte verkleinen.** Laad zware delen later (`@defer`, `loadComponent`) en gebruik alleen wat je nodig hebt. Het is een goede gewoonte om bij elke grote bibliotheek te kijken wat ze kost.

Lokaal testen zoals in productie

```
npx serve -s dist/weekmenu/browser
```

De optie `-s` zorgt dat elke onbekende URL `index.html` teruggeeft. Dat is nodig voor een **single-page app**: gaat iemand direct naar `/weekplanner`, dan bestaat dat bestand niet op de server, en Angular moet de route afhandelen. Zonder deze regel krijg je een 404.

MODULE 39: Server-side rendering (SSR): wat, wanneer, en niet voor deze app

Tot nu toe is Weekmenu een **client-side** app: de browser downloadt een bijna lege `index.html` plus JavaScript, en Angular tekent alles in de browser. Bij **SSR** rendert een server de pagina al tot HTML, zodat:

- zoekmachines en deelkaarten (Open Graph) echte inhoud zien,
- de eerste weergave sneller is op trage apparaten,
- Angular daarna de pagina "overneemt" (**hydration**) in plaats van opnieuw te tekenen.

Je zet het aan met:

```
npx ng add @angular/ssr
```

Dat voegt een server en configuratie toe: `src/server.ts`, `src/main.server.ts`, `src/app/app.config.server.ts` en `src/app/app.routes.server.ts`. In dat laatste bestand kies je per route hoe hij gerenderd wordt: `RenderMode.Server` (bij elk bezoek op de server), `RenderMode.Prerender` (vooraf tijdens de build, de standaard) of `RenderMode.Client` (alleen in de browser).

> **Let op:** krijg je bij `ng add` een foutmelding over conflicterende versies (`ERESOLVE`), dan hebben je `@angular/...`-pakketten niet precies dezelfde versie. Werk ze bij met `npx ng update @angular/core @angular/cli` en probeer het opnieuw. Een nieuw project met SSR maak je ook rechtstreeks met `npx @angular/cli@latest new naam --ssr`.

Waarom gebruiken we het niet in Weekmenu?

- Alle pagina's zitten **achter een login**. Zoekmachines komen er toch niet. Bovendien kent een server de sessie niet: die staat in `localStorage` van de browser. Onze `authGuard` zou op de server altijd naar het inloggen doorsturen.
- Code die `localStorage`, `window` of `navigator` gebruikt, werkt niet op de server. Je moet die in `afterNextRender` zetten of controleren met `isPlatformBrowser`.

Wanneer wel? Voor openbare websites: een bedrijfssite, een blog, een webshop met productpagina's. Dan is SSR (of prerendering) de moeite waard. Voor een app achter een login is een gewone client-side app prima.

> **Vergelijking:** dit is hetzelfde verschil als tussen een Vue- of React-SPA en Nuxt of Next.js. Angular's SSR is een optionele laag die je later kunt toevoegen.

MODULE 40: Deployen naar Vercel

Weekmenu is nu een statische site: een map met bestanden. Die kun je op elke statische host zetten. We gebruiken Vercel, gekoppeld aan GitHub.

Stap 1: Het project klaarmaken

Voeg aan `package.json` de Node-versie toe, zodat Vercel dezelfde Node gebruikt als jij (Angular 22 vraagt een recente versie). Voeg dit blok toe op het hoogste niveau van het bestand (naast `"scripts"`):

```
"engines": {
  "node": ">=22.22.3"
}
```

Maak in de root van het project een bestand `vercel.json`, zodat elke URL naar `index.html` gaat (Angular regelt de routes zelf):

```
{
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]
}
```

Toelichting: Vercel serveert eerst een bestaand bestand (zoals `main-ABC123.js`) en valt pas daarna terug op `index.html`. Daardoor gaan assets niet kapot.

Controleer dat alles werkt en leg het vast:

```
npm run build
npx ng test --no-watch
git add .
git commit -m "Weekmenu: eerste versie"
```

(`ng new` heeft al een Git-repository aangemaakt.)

Stap 2: Naar GitHub

1. Maak op github.com een **nieuwe lege repository** (bijvoorbeeld `weekmenu`).
2. Koppel en push (vervang de URL door die van jou):

```
git branch -M main
git remote add origin https://github.com/JOUW-NAAM/weekmenu.git
git push -u origin main
```

Stap 3: Vercel

1. Log in op vercel.com met je GitHub-account en kies **Add New, Project**.
2. Importeer de repository `weekmenu`. Vercel herkent Angular.
3. Controleer de instellingen: - **Build Command:** `npm run build` - **Output Directory:** `dist/weekmenu/browser`
4. Klik op **Deploy**.

Je krijgt een adres als `https://weekmenu.vercel.app`.

Stap 4: Supabase laten weten waar je app staat

Bij het inloggen en bevestigingsmails gebruikt Supabase een **Site URL** en een lijst toegestane **redirect-URL's**. Open in het Supabase-dashboard onder **Authentication** de URL-instellingen en voeg je Vercel-adres toe (naast `http://localhost:4200`). Doe je dit niet, dan verwijzen bevestigingsmails naar het verkeerde adres.

Stap 5: Controleren

- Open je Vercel-adres, registreer en log in.
- Ga rechtstreeks naar `/weekplanner` in de adresbalk: de pagina moet laden (dat test de `rewrites`).
- Open de ontwikkelaarstools, tab Netwerk: je ziet de verzoeken naar Supabase met de juiste headers.

Over geheimen in een Angular-app

- **Alles wat in je Angular-code staat, is publiek.** De `environment`-bestanden komen als gewone tekst in de gebouwde JavaScript.
- Daarom staat hier alleen de **publishable sleutel**, die bedoeld is voor de browser. De beveiliging zit in **RLS** in de database.
- Heb je ooit een geheim nodig (een betaal- of e-mailsleutel)? Dan hoort dat op een **server** (een Supabase Edge Function of een aparte backend), niet in Angular.

Elke push is een nieuwe deployment

Een `git push` naar `main` start automatisch een nieuwe productie-deployment. Een push naar een andere branch maakt een **preview-deployment** met een eigen adres.

Bijlage A: Oudere Angular-code herkennen

De meeste bestaande Angular-projecten (en veel tutorials) gebruiken oudere patronen. Ze werken nog steeds en je komt ze op een stage of in je eerste baan zeker tegen. Zo vertaal je ze naar wat je in deze cursus leerde.

Oud (tot ongeveer Angular 16)	Nieuw (deze cursus)
`@NgModule({ declarations, imports })`	standalone componenten met `imports`
`recipe-card.component.ts`, klasse `RecipeCardComponent`	`recipe-card.ts`, klasse `RecipeCard`
`@Input() recipe!: Recipe;`	`recipe = input.required<Recipe>()`
`@Output() open = new EventEmitter<number>();`	`open = output<number>()`
`*ngIf="x"`, `*ngIf="x; else other"`	`@if (x) { } @else { }`
`*ngFor="let r of list; trackBy: fn"`	`@for (r of list; track r.id) { }`
`[ngSwitch]` en `*ngSwitchCase`	`@switch` en `@case`
`constructor(private store: RecipeStore) {}`	`private readonly store = inject(RecipeStore);`
`@Injectable({ providedIn: 'root' })`	`@Service()`
`ngOnInit() { ... }`	de constructor, `computed` of `effect`
`ngOnChanges(changes)`	`computed`, `linkedSignal` of `effect`
`ngOnDestroy()`	`inject(DestroyRef).onDestroy(...)`
`this.route.snapshot.paramMap.get('id')`	`id = input.required<string>()` met `withComponentInputBinding()`
`this.route.paramMap.subscribe(...)`	idem
`BehaviorSubject<T>` voor state in een service	`signal<T>()`
`observable$ | async` in de template	`toSignal(observable$)` of `httpResource`
`HttpClientModule` in een module	`provideHttpClient()` in `app.config.ts`
`HttpInterceptor` (klasse)	`HttpInterceptorFn` (functie)
`CanActivate` (klasse)	`CanActivateFn` (functie)
`[(ngModel)]="naam"` (template-driven)	Signal Forms of Reactive Forms
`@HostListener('click')`, `@HostBinding('class')`	`host: { '(click)': '...', '[class]': '...' }`
`@ViewChild('ref') el!: ElementRef;`	`el = viewChild.required<ElementRef>('ref')`
`zone.js` en `ChangeDetectionStrategy.OnPush`	zoneless met signals (nieuwe projecten)
`ng-content`	hetzelfde
NgRx (Store, Actions, Reducers, Effects)	voor grote apps nog veel gebruikt; zie "Wat nu?"

Een belangrijk detail: `*ngIf` en `*ngFor` zijn niet verdwenen. Je ziet ze overal in bestaande code, en de CLI kan ze automatisch omzetten (`ng generate @angular/core:control-flow`). Leer ze lezen; schrijf in nieuwe code `@if` en `@for`.

Bijlage B: Vue/React naar Angular: spiekbriefje

Concept	Vue 3	React 19	Angular 22
Component	`.vue`-bestand	functie met JSX	klasse met `@Component` en een template
Toestand	`ref(0)`	`useState(0)`	`signal(0)`
Lezen	`count.value`	`count`	`count()`
Wijzigen	`count.value++`	`setCount(c + 1)`	`count.update((c) => c + 1)`
Afgeleide waarde	`computed()`	tijdens het renderen berekenen	`computed()`
Volgen en reageren	`watch`, `watchEffect`	`useEffect`	`effect()`
Beginwaarde die reset	`watch` + `ref`	`key`-truc	`linkedSignal()`
Props	`defineProps`	props-object	`input()` en `input.required()`
Events naar boven	`defineEmits`	callback-prop	`output()`
Slots	`<slot />`	`children`	`<ng-content />`
Voorwaardelijk	`v-if`	`cond && <A />`	`@if`
Lijsten	`v-for :key`	`map` met `key`	`@for` met `track`
Tekst tonen	`{{ x }}`	`{x}`	`{{ x() }}`
Attribuut binden	`:prop="x"`	`prop={x}`	`[prop]="x()"`
Event	`@click="f"`	`onClick={f}`	`(click)="f()"`
Tweerichtingsbinding	`v-model`	`value` + `onChange`	`[formField]` of `formControlName`
Gedeelde state	Pinia	Context en reducers	service met signals
Dependency injection	`provide` / `inject`	Context	`inject()` met `@Service()`
Router	Vue Router	React Router, of Next.js	`@angular/router` met een routelijst
Beschermde routes	`beforeEach`	wrapper-component	`canActivate`-guards
HTTP	`fetch`, Axios	`fetch`, TanStack Query	`HttpClient`, `httpResource`
HTTP middleware	Axios-interceptors	n.v.t.	interceptors
Asynchrone stromen	`watch`, composables	effecten	RxJS
Lazy loading	`defineAsyncComponent`	`React.lazy`	`loadComponent` en `@defer`
Pipes/filters	methoden	helperfuncties	`| pipe`
Eigen DOM-gedrag	`v-custom` directives	refs en effecten	directives
Tests	Vitest + Vue Test Utils	Vitest/Jest + Testing Library	Vitest + `TestBed`
Opmaak	los CSS of Tailwind	idem	idem

Bijlage C: Veelgemaakte fouten

1. **Een signal lezen zonder haakjes.** `{{ count }}` toont de functie, niet de waarde. Schrijf `{{ count() }}`.
2. **Een signal wijzigen door zijn inhoud aan te passen** (`list().push(x)`, `set.add(x)`). Angular merkt het niet. Maak een nieuwe array, set of object met `update` of `set`.
3. **Een component gebruiken zonder hem in `imports` te zetten.** Foutmelding: *"is not a known element"*. Voeg de klasse toe aan `imports` van het component dat hem gebruikt.
4. **`@for` zonder `track`.** Is verplicht. Gebruik een stabiele sleutel zoals een id.
5. **Een Observable maken en niet abonneren.** Een `http.post(...)` doet **niets** zonder `subscribe`, `firstValueFrom` of een `async`-pipe.
6. **Handmatig `subscribe` zonder opruimen.** Gebruik `toSignal`, de `async`-pipe, of `takeUntilDestroyed()` om lekken te voorkomen.
7. **`effect` gebruiken om een waarde af te leiden.** Gebruik `computed` of `linkedSignal`.
8. **Routevolgorde.** Een specifieke route (`recepten/nieuw`) moet vóór een parameterroute (`recepten/:id`) staan, en `**` helemaal onderaan.
9. **`inject()` buiten een injectiecontext.** `inject` werkt alleen in een constructor, een veldinitialisatie, of een functie die Angular zelf uitvoert (guards, interceptors). Niet in een `setTimeout` of een `then`.
10. **`providedIn` of `@Service` vergeten, of een service per ongeluk meerdere keren aanmaken.** Als je data niet gedeeld lijkt, controleer of de service niet in `providers` van een component staat.
11. **Een geheime sleutel in de Angular-code.** Alles in je app is publiek. Alleen de publishable sleutel hoort er te staan.
12. **RLS vergeten.** Zonder policies kan niemand iets lezen; met `using (true)` kan iedereen alles lezen. Test altijd met een tweede account.
13. **Ongecontroleerde invoer uit de URL gebruiken.** Parameters zijn altijd tekst en kunnen onzin zijn. Zet ze om en controleer ze (zoals `terug` bij het inloggen).
14. **`localStorage` gebruiken in code die ook op een server draait** (alleen bij SSR). Zet het in `afterNextRender` of controleer `isPlatformBrowser`.
15. **Geen `await fixture.whenStable()` in een test.** De template is dan nog niet bijgewerkt.

Bijlage D: Eindopdrachten en wat je daarna leert

Eindopdrachten

1. **Recepten zoeken op ingrediënt.** Voeg een zoekveld toe dat ook in de ingrediënten zoekt. Gebruik `toSignal` met `debounceTime`.
2. **Favorieten.** Voeg een kolom `favorite` toe en een hartje op de kaart. Gebruik een `PATCH`-verzoek en werk de store lokaal bij.
3. **Foto's.** Sla een foto per recept op in Supabase Storage en toon hem met `NgOptimizedImage` (`<img ngSrc>`).
4. **Weekplanning per aantal personen.** Voeg `servings` toe aan de planning en schaal de boodschappenlijst.
5. **Een dashboard.** Een pagina met statistieken: aantal recepten per categorie, de meest geplande recepten, de gemiddelde bereidingstijd. Gebruik `computed`.
6. **Gedeelde recepten.** Maak een tabel `shared_recipes` met een openbare leespolicy en een aparte pagina zonder login. Dit is een goede reden om SSR te proberen.
7. **Wachtwoord vergeten.** Voeg een pagina toe met `auth.resetPasswordForEmail` en een pagina waar de gebruiker een nieuw wachtwoord instelt.
8. **Meer tests.** Test `RecipeForm`: een leeg formulier is ongeldig, een ingevuld formulier roept `store.add` aan.
9. **Toegankelijkheid.** Controleer de app met het toetsenbord en met Lighthouse. Voeg waar nodig `aria`-attributen toe.

Wat leer je daarna?

Onderwerp	Waarom
RxJS verdiepen (`switchMap`, `combineLatest`, `retry`)	bijna elk bestaand Angular-project gebruikt het intensief
NgRx of **NgRx SignalStore**	gestructureerde state voor grote applicaties
Angular Material en CDK	kant-en-klare componenten en bouwstenen (dialogen, overlays, drag-and-drop)
Interne structuur van grote projecten (feature folders, lazy-loaded features, Nx)	schaalbare code en monorepo's
Tests met Playwright	end-to-end tests in een echte browser
Formulieren verdiepen (eigen validators, dynamische formulieren)	veel zakelijke apps zijn formulieren
i18n (`@angular/localize`)	meertalige apps
SSR en prerendering	voor openbare websites
Prestaties (`@defer`, `NgOptimizedImage`, de Angular DevTools)	snelle apps

Bijlage E: Overzicht van het eindresultaat

```
weekmenu/
  angular.json
  package.json
  vercel.json
  src/
    index.html
    main.ts
    styles.css
    environments/
      environment.ts
      environment.development.ts
    app/
      app.ts                        shell met navigatie en uitloggen
      app.html
      app.config.ts                 router, HTTP en interceptors
      app.routes.ts                 routes met guards en lazy loading
      auth/
        login/                      Reactive Forms
      core/
        api.ts                      REST-adres
        auth-store.ts               sessie met signals (Supabase Auth)
        auth-guard.ts               authGuard en guestGuard
        supabase-interceptor.ts     apikey en token
        error-interceptor.ts        401 afhandelen
      recipes/
        recipe.ts                   types en rekenlogica
        recipe-store.ts             httpResource en HttpClient
        recipe-card/
        recipe-list/                zoeken (RxJS), filter, effect
        recipe-detail/              linkedSignal, @defer
        recipe-form/                Signal Forms
        recipe-steps/
      planner/
        meal-plan.ts                datums en boodschappenlogica
        plan-store.ts
        planner/
      shopping/
        shopping-list/
      shared/
        panel/                      content projection
        form-errors/
        not-found/
        duration-pipe.ts
        copy.ts                     attribuut-directive
```

Bijlage F: Commando-overzicht

Doel	Commando
Node en npm controleren	`node -v` en `npm -v`
Nieuw project	`npx @angular/cli@latest new naam --style=tailwind --routing --ssr=false`
Ontwikkelserver	`npm start` of `npx ng serve`
Component genereren	`npx ng generate component map/naam --skip-tests --style=none`
Service genereren	`npx ng generate service map/naam`
Guard, interceptor, pipe, directive	`npx ng generate guard` / `interceptor` / `pipe` / `directive`
Environments genereren	`npx ng generate environments`
Tests	`npx ng test` (of `--no-watch`)
Productiebuild	`npm run build`
Productiebuild lokaal bekijken	`npx serve -s dist/weekmenu/browser`
Oude syntax omzetten naar `@if` en `@for`	`npx ng generate @angular/core:control-flow`
Angular bijwerken	`npx ng update @angular/core @angular/cli`
Server stoppen	`Ctrl + C`

Einde van de cursus. Succes met bouwen!